AI Impact on Jobs & Layoff Risk Analysis

Using Machine Learning

Complete Project Code

All 3 Phases (Month-1, Month-2, Month-3)

GTU Semester-5

BE05000231 - Python for Data Science

Student Details

Name: Sharma Saurabh

Enrollment: 250863131007

Project Duration: 3 Months

Status: Completed

Table of Contents

Month-1: Foundation Phase

1. config.py - Project Configuration
2. requirements.txt - Dependencies
3. main.py - Main Entry Point
4. src/__init__.py - Package Initializer
5. src/data_loading.py - Data Loading Module
6. src/data_cleaning.py - Data Cleaning Module

Month-2: Analysis Phase

7. src/exploratory_data_analysis.py - EDA Module
8. src/feature_engineering.py - Feature Engineering
9. src/visualization.py - Data Visualization
10. src/preprocessing.py - Data Preprocessing

Month-3: Modeling Phase

11. src/model_training.py - Model Training
12. src/model_evaluation.py - Model Evaluation
13. src/prediction.py - Prediction System
14. src/utils.py - Utility Functions
15. README.md - Project Documentation

config.py

```
"""
Configuration Module for AI Impact on Jobs & Layoff Risk Analysis Project.

This module centralizes all project paths and configuration settings.
It ensures no hardcoded paths are used throughout the project.

Author: Sharma Saurabh (250863131007)
Course: BE05000231 - Python for Data Science (GTU Semester-5)
"""

import os

# ============================================================================
# PROJECT ROOT PATH
# ============================================================================

PROJECT_ROOT = os.path.dirname(os.path.abspath(__file__))

# ============================================================================
# DATA FOLDER PATHS
# ============================================================================

DATA_DIR = os.path.join(PROJECT_ROOT, "data")
RAW_DATA_DIR = os.path.join(DATA_DIR, "raw")
PROCESSED_DATA_DIR = os.path.join(DATA_DIR, "processed")

# ============================================================================
# RAW DATASET PATH
# ============================================================================

RAW_DATASET_PATH = os.path.join(
    RAW_DATA_DIR, "ai-impact-jobs-layoff-risk-dataset.csv"
)

# ============================================================================
# OUTPUT FOLDER PATHS
# ============================================================================

OUTPUTS_DIR = os.path.join(PROJECT_ROOT, "outputs")
PROCESSED_OUTPUT_DIR = os.path.join(OUTPUTS_DIR, "processed_data")
CHARTS_OUTPUT_DIR = os.path.join(OUTPUTS_DIR, "charts")
MODELS_OUTPUT_DIR = os.path.join(OUTPUTS_DIR, "models")
REPORTS_OUTPUT_DIR = os.path.join(OUTPUTS_DIR, "reports")
PREDICTIONS_OUTPUT_DIR = os.path.join(OUTPUTS_DIR, "predictions")

# ============================================================================
# FILE PATHS
# ============================================================================

CLEANED_DATASET_PATH = os.path.join(PROCESSED_OUTPUT_DIR, "cleaned_dataset.csv")
ENGINEERED_DATASET_PATH = os.path.join(PROCESSED_OUTPUT_DIR, "engineered_dataset.csv")
PREPROCESSED_DATASET_PATH = os.path.join(PROCESSED_OUTPUT_DIR, "preprocessed_dataset.csv")
TRAIN_DATA_PATH = os.path.join(PROCESSED_OUTPUT_DIR, "train_data.csv")
TEST_DATA_PATH = os.path.join(PROCESSED_OUTPUT_DIR, "test_data.csv")
BEST_MODEL_PATH = os.path.join(MODELS_OUTPUT_DIR, "best_model.pkl")
SCALER_PATH = os.path.join(MODELS_OUTPUT_DIR, "scaler.pkl")
ENCODER_PATH = os.path.join(MODELS_OUTPUT_DIR, "encoder.pkl")
EVALUATION_REPORT_PATH = os.path.join(REPORTS_OUTPUT_DIR, "evaluation_report.txt")
PREDICTIONS_PATH = os.path.join(PREDICTIONS_OUTPUT_DIR, "predictions.csv")

# ============================================================================
# DOCUMENTATION FOLDER PATH
# ============================================================================

DOCS_DIR = os.path.join(PROJECT_ROOT, "docs")

# ============================================================================
# TESTS FOLDER PATH
# ============================================================================

TESTS_DIR = os.path.join(PROJECT_ROOT, "tests")
```

```
# ============================================================================
# MODEL PARAMETERS
# ============================================================================

TEST_SIZE = 0.2
RANDOM_STATE = 42
TARGET_COLUMN = "layoff_risk"
```

requirements.txt

```
pandas>=1.5.0
numpy>=1.23.0
matplotlib>=3.6.0
seaborn>=0.12.0
scikit-learn>=1.1.0
joblib>=1.2.0
```

main.py

```
"""
Main Entry Point for AI Impact on Jobs & Layoff Risk Analysis Project.

This module orchestrates the complete machine learning pipeline:
- Month-1: Data Loading, Inspection, Cleaning
- Month-2: EDA, Feature Engineering, Visualization, Preprocessing
- Month-3: Model Training, Evaluation, Prediction

Author: Sharma Saurabh (250863131007)
Course: BE05000231 - Python for Data Science (GTU Semester-5)

Usage:
    python main.py
"""

import sys
import os

# Add project root to path for imports
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

import config
from src.data_loading import (
    load_dataset,
    inspect_dataset,
    display_shape,
    display_columns,
    display_data_types,
    display_missing_values,
)
from src.data_cleaning import (
    handle_missing_values,
    remove_duplicates,
    remove_invalid_records,
    standardize_column_names,
    save_clean_dataset,
)
from src.exploratory_data_analysis import run_complete_eda
from src.feature_engineering import run_feature_engineering
from src.visualization import run_complete_visualization
from src.preprocessing import run_preprocessing
from src.model_training import run_model_training
from src.model_evaluation import evaluate_all_models, save_evaluation_report
from src.prediction import load_model, load_preprocessor, run_predictions
from src.utils import print_section_header, print_subsection_header, get_timestamp


def print_banner():
    """Print the project banner."""
    print("\n" + "=" * 70)
    print("=" * 70)
    print("  AI IMPACT ON JOBS & LAYOFF RISK ANALYSIS")
    print("  Using Machine Learning")
    print("  GTU Semester-5: Python for Data Science (BE05000231)")
    print("  Student: Sharma Saurabh (250863131007)")
    print("=" * 70)
    print("=" * 70)


def run_month1():
    """
    Run Month-1: Foundation Phase.

    Returns:
        pandas.DataFrame: Cleaned DataFrame.
    """
    print_section_header("MONTH-1: FOUNDATION PHASE")

    # Phase 1: Data Loading
    print_subsection_header("PHASE 1: DATA LOADING")
    df = load_dataset(config.RAW_DATASET_PATH)
```

```
    # Phase 2: Dataset Inspection
    print_subsection_header("PHASE 2: DATASET INSPECTION")
    rows, cols = display_shape(df)
    columns = display_columns(df)
    dtypes = display_data_types(df)
    missing = display_missing_values(df)
    inspect_dataset(df)

    # Phase 3: Data Cleaning
    print_subsection_header("PHASE 3: DATA CLEANING")
    df = handle_missing_values(df, strategy="drop")
    df = remove_duplicates(df)
    df = remove_invalid_records(df)
    df = standardize_column_names(df)

    # Phase 4: Save Cleaned Dataset
    print_subsection_header("PHASE 4: SAVE CLEANED DATASET")
    save_clean_dataset(df, config.CLEANED_DATASET_PATH)

    print(f"\n[Month-1 Complete] Cleaned dataset: {df.shape[0]} rows x {df.shape[1]} columns")

    return df


def run_month2(df):
    """
    Run Month-2: Analysis Phase.

    Parameters:
        df (pandas.DataFrame): Cleaned DataFrame from Month-1.

    Returns:
        pandas.DataFrame: Preprocessed DataFrame ready for ML.
    """
    print_section_header("MONTH-2: ANALYSIS PHASE")

    # Phase 1: Exploratory Data Analysis
    print_subsection_header("PHASE 1: EXPLORATORY DATA ANALYSIS")
    eda_results = run_complete_eda(df)

    # Phase 2: Feature Engineering
    print_subsection_header("PHASE 2: FEATURE ENGINEERING")
    df_engineered = run_feature_engineering(df)

    # Phase 3: Data Visualization
    print_subsection_header("PHASE 3: DATA VISUALIZATION")
    run_complete_visualization(df_engineered, output_dir=config.CHARTS_OUTPUT_DIR)

    # Phase 4: Preprocessing
    print_subsection_header("PHASE 4: DATA PREPROCESSING")
    X_train, X_test, y_train, y_test, scaler, encoders = run_preprocessing(
        df_engineered, config
    )

    print(f"\n[Month-2 Complete] Training set: {X_train.shape}, Testing set: {X_test.shape}")

    return X_train, X_test, y_train, y_test, scaler, encoders


def run_month3(X_train, X_test, y_train, y_test, scaler, encoders):
    """
    Run Month-3: Modeling Phase.

    Parameters:
        X_train, X_test: Training and testing features.
        y_train, y_test: Training and testing labels.
        scaler: Fitted scaler.
        encoders: Dictionary of encoders.
    """
    print_section_header("MONTH-3: MODELING PHASE")

    # Phase 1: Model Training
    print_subsection_header("PHASE 1: MODEL TRAINING")
```

```
    trained_models, comparison_df, best_model_name = run_model_training(
        X_train, X_test, y_train, y_test, config
    )

    # Phase 2: Model Evaluation
    print_subsection_header("PHASE 2: MODEL EVALUATION")
    all_results = evaluate_all_models(
        trained_models, X_test, y_test, output_dir=config.CHARTS_OUTPUT_DIR
    )

    # Save evaluation report
    save_evaluation_report(all_results, config.EVALUATION_REPORT_PATH)

    # Phase 3: Predictions
    print_subsection_header("PHASE 3: MAKING PREDICTIONS")
    best_model = trained_models[best_model_name]["model"]
    predictions_df = run_predictions(
        best_model, scaler, encoders, X_test, y_test, config
    )

    print(f"\n[Month-3 Complete] Best model: {best_model_name}")


def print_completion_summary():
    """Print final completion summary."""
    print("\n" + "=" * 70)
    print("=" * 70)
    print("  PROJECT COMPLETED SUCCESSFULLY!")
    print("")
    print("  Phases Completed:")
    print("  [Month-1] Foundation Phase")
    print("    - Project structure created")
    print("    - Dataset loaded and inspected")
    print("    - Data cleaned and saved")
    print("")
    print("  [Month-2] Analysis Phase")
    print("    - Exploratory Data Analysis performed")
    print("    - Features engineered")
    print("    - Visualizations generated")
    print("    - Data preprocessed for ML")
    print("")
    print("  [Month-3] Modeling Phase")
    print("    - Multiple ML models trained")
    print("    - Models evaluated and compared")
    print("    - Best model saved")
    print("    - Predictions generated")
    print("")
    print("  Output Files:")
    print(f"    - Cleaned Data: {config.CLEANED_DATASET_PATH}")
    print(f"    - Charts: {config.CHARTS_OUTPUT_DIR}")
    print(f"    - Best Model: {config.BEST_MODEL_PATH}")
    print(f"    - Evaluation Report: {config.EVALUATION_REPORT_PATH}")
    print(f"    - Predictions: {config.PREDICTIONS_PATH}")
    print("=" * 70)
    print("=" * 70 + "\n")


def main():
    """
    Main function to execute the complete project pipeline.

    This function orchestrates all three phases:
    1. Month-1: Foundation Phase
    2. Month-2: Analysis Phase
    3. Month-3: Modeling Phase

    Returns:
        None
    """
    try:
        # Print banner
        print_banner()
        print(f"\nPipeline started at: {get_timestamp()}")
```

```
        # Run Month-1
        df = run_month1()

        # Run Month-2
        X_train, X_test, y_train, y_test, scaler, encoders = run_month2(df)

        # Run Month-3
        run_month3(X_train, X_test, y_train, y_test, scaler, encoders)

        # Print completion summary
        print_completion_summary()

        print(f"Pipeline completed at: {get_timestamp()}")

    except KeyboardInterrupt:
        print("\n\n[INFO] Process interrupted by user.")
        sys.exit(0)
    except Exception as e:
        print(f"\n[FATAL ERROR] {e}")
        import traceback
        traceback.print_exc()
        sys.exit(1)


if __name__ == "__main__":
    main()
```

src/__init__.py

```
"""
Source Package for AI Impact on Jobs & Layoff Risk Analysis Project.

This package contains all source modules for data loading, cleaning,
and processing operations.

Author: BE05000231 - Python for Data Science (GTU Semester-5)
Month: 1 - Foundation Phase
"""
```

src/data_loading.py

```
"""
Data Loading Module for AI Impact on Jobs & Layoff Risk Analysis Project.

This module provides functions to load, inspect, and validate the dataset.
It handles file existence checks, data loading using pandas, and provides
comprehensive dataset inspection capabilities.

Author: BE05000231 - Python for Data Science (GTU Semester-5)
Month: 1 - Foundation Phase
"""

import os
import pandas as pd
import sys


def check_file_exists(file_path):
    """
    Check if the specified file exists at the given path.

    This function validates the existence of a file before attempting
    to load it, preventing runtime errors.

    Parameters:
        file_path (str): The full path to the file to check.

    Returns:
        bool: True if file exists, False otherwise.

    Examples:
        >>> check_file_exists("data/raw/dataset.csv")
        True
        >>> check_file_exists("data/raw/nonexistent.csv")
        False
    """
    try:
        if os.path.exists(file_path):
            print(f"[OK] File found: {file_path}")
            return True
        else:
            print(f"[ERROR] File not found: {file_path}")
            return False
    except Exception as e:
        print(f"[ERROR] Error checking file existence: {e}")
        return False


def load_dataset(file_path):
    """
    Load the CSV dataset into a pandas DataFrame.

    This function reads a CSV file and returns a pandas DataFrame.
    It includes error handling for common loading issues.

    Parameters:
        file_path (str): The full path to the CSV file.

    Returns:
        pandas.DataFrame: The loaded dataset as a DataFrame.

    Raises:
        FileNotFoundError: If the file does not exist.
        pd.errors.EmptyDataError: If the file is empty.
        pd.errors.ParserError: If the file cannot be parsed as CSV.

    Examples:
        >>> df = load_dataset("data/raw/dataset.csv")
        >>> print(type(df))
        <class 'pandas.core.frame.DataFrame'>
    """
    try:
```

```
        if not check_file_exists(file_path):
            raise FileNotFoundError(f"Dataset not found at: {file_path}")

        print(f"\nLoading dataset from: {file_path}")
        df = pd.read_csv(file_path)
        print(f"[OK] Dataset loaded successfully!")
        return df

    except FileNotFoundError as e:
        print(f"[ERROR] FileNotFoundError: {e}")
        sys.exit(1)
    except pd.errors.EmptyDataError:
        print(f"[ERROR] The file is empty: {file_path}")
        sys.exit(1)
    except pd.errors.ParserError as e:
        print(f"[ERROR] ParserError: Unable to parse CSV file. {e}")
        sys.exit(1)
    except Exception as e:
        print(f"[ERROR] Unexpected error while loading dataset: {e}")
        sys.exit(1)


def inspect_dataset(df):
    """
    Display basic information about the dataset.

    This function prints the first and last few rows of the dataset
    along with basic statistics to help understand the data structure.

    Parameters:
        df (pandas.DataFrame): The DataFrame to inspect.

    Returns:
        None

    Examples:
        >>> df = load_dataset("data/raw/dataset.csv")
        >>> inspect_dataset(df)
    """
    try:
        if df is None or df.empty:
            print("[WARNING] DataFrame is None or empty.")
            return

        print("\n" + "=" * 60)
        print("DATASET INSPECTION")
        print("=" * 60)

        print("\n--- First 5 Rows ---")
        print(df.head())

        print("\n--- Last 5 Rows ---")
        print(df.tail())

        print("\n--- Basic Statistics ---")
        print(df.describe())

        print("\n" + "=" * 60)

    except Exception as e:
        print(f"[ERROR] Error inspecting dataset: {e}")


def display_shape(df):
    """
    Display the shape (rows, columns) of the dataset.

    Parameters:
        df (pandas.DataFrame): The DataFrame to get shape from.

    Returns:
        tuple: A tuple containing (number_of_rows, number_of_columns).

    Examples:
```

```
        >>> rows, cols = display_shape(df)
        >>> print(f"Dataset has {rows} rows and {cols} columns")
    """
    try:
        if df is None or df.empty:
            print("[WARNING] DataFrame is None or empty.")
            return (0, 0)

        rows, cols = df.shape
        print(f"\nDataset Shape: {rows} rows x {cols} columns")
        return (rows, cols)

    except Exception as e:
        print(f"[ERROR] Error getting dataset shape: {e}")
        return (0, 0)


def display_columns(df):
    """
    Display all column names in the dataset.

    Parameters:
        df (pandas.DataFrame): The DataFrame to list columns from.

    Returns:
        list: A list of column names.

    Examples:
        >>> columns = display_columns(df)
        >>> print(columns)
        ['Age', 'Education_Level', 'Industry', ...]
    """
    try:
        if df is None or df.empty:
            print("[WARNING] DataFrame is None or empty.")
            return []

        columns = df.columns.tolist()
        print(f"\nColumn Names ({len(columns)} columns):")
        for i, col in enumerate(columns, 1):
            print(f"  {i}. {col}")
        return columns

    except Exception as e:
        print(f"[ERROR] Error displaying columns: {e}")
        return []


def display_data_types(df):
    """
    Display the data type of each column in the dataset.

    Parameters:
        df (pandas.DataFrame): The DataFrame to inspect.

    Returns:
        pandas.Series: A Series containing column names and their data types.

    Examples:
        >>> dtypes = display_data_types(df)
        >>> print(dtypes)
    """
    try:
        if df is None or df.empty:
            print("[WARNING] DataFrame is None or empty.")
            return None

        dtypes = df.dtypes
        print("\nColumn Data Types:")
        print("-" * 40)
        for col, dtype in dtypes.items():
            print(f"  {col}: {dtype}")
        return dtypes
```

```
    except Exception as e:
        print(f"[ERROR] Error displaying data types: {e}")
        return None


def display_missing_values(df):
    """
    Display the count and percentage of missing values in each column.

    This function helps identify columns with missing data that need
    to be handled during the data cleaning phase.

    Parameters:
        df (pandas.DataFrame): The DataFrame to check for missing values.

    Returns:
        pandas.Series: A Series containing missing value counts per column.

    Examples:
        >>> missing = display_missing_values(df)
        >>> print(missing)
    """
    try:
        if df is None or df.empty:
            print("[WARNING] DataFrame is None or empty.")
            return None

        missing_count = df.isnull().sum()
        missing_percent = (missing_count / len(df)) * 100

        print("\nMissing Values Analysis:")
        print("-" * 50)
        print(f"{'Column':<35} {'Missing':<10} {'Percent':<10}")
        print("-" * 50)

        has_missing = False
        for col, count in missing_count.items():
            if count > 0:
                has_missing = True
                percent = missing_percent[col]
                print(f"  {col:<33} {count:<10} {percent:.2f}%")

        if not has_missing:
            print("  No missing values found in any column!")

        print("-" * 50)
        total_missing = missing_count.sum()
        print(f"\nTotal Missing Values: {total_missing}")

        return missing_count

    except Exception as e:
        print(f"[ERROR] Error displaying missing values: {e}")
        return None
```

src/data_cleaning.py

```
"""
Data Cleaning Module for AI Impact on Jobs & Layoff Risk Analysis Project.

This module provides functions to clean and preprocess the dataset.
It handles missing values, duplicate removal, invalid record detection,
column name standardization, and saving cleaned data.

Author: BE05000231 - Python for Data Science (GTU Semester-5)
Month: 1 - Foundation Phase
"""

import os
import pandas as pd
import re


def handle_missing_values(df, strategy="drop", threshold=0.5):
    """
    Handle missing values in the dataset.

    This function addresses missing data using various strategies
    to ensure data quality for subsequent analysis.

    Parameters:
        df (pandas.DataFrame): The DataFrame to clean.
        strategy (str): The strategy to handle missing values.
            - "drop": Remove rows with any missing values.
            - "fill_mean": Fill numeric columns with mean.
            - "fill_median": Fill numeric columns with median.
            - "fill_mode": Fill all columns with mode.
        threshold (float): Maximum percentage of missing values allowed
            in a column (0.0 to 1.0). Columns exceeding this threshold
            are dropped entirely.

    Returns:
        pandas.DataFrame: DataFrame with missing values handled.

    Examples:
        >>> df = handle_missing_values(df, strategy="drop")
        >>> df = handle_missing_values(df, strategy="fill_median")
    """
    try:
        if df is None or df.empty:
            print("[WARNING] DataFrame is None or empty.")
            return df

        print("\n" + "=" * 60)
        print("HANDLING MISSING VALUES")
        print("=" * 60)

        initial_rows = len(df)
        missing_before = df.isnull().sum().sum()
        print(f"\nMissing values before cleaning: {missing_before}")

        # Drop columns with missing values exceeding threshold
        missing_percent = df.isnull().mean()
        cols_to_drop = missing_percent[missing_percent > threshold].index.tolist()

        if cols_to_drop:
            print(f"\nDropping columns with >{threshold*100}% missing values:")
            for col in cols_to_drop:
                print(f"  - {col} ({missing_percent[col]*100:.2f}% missing)")
            df = df.drop(columns=cols_to_drop)

        # Apply strategy for remaining missing values
        if strategy == "drop":
            rows_before = len(df)
            df = df.dropna()
            rows_dropped = rows_before - len(df)
            print(f"\nStrategy: Drop rows with missing values")
            print(f"Rows dropped: {rows_dropped}")
```

```
        elif strategy == "fill_mean":
            print(f"\nStrategy: Fill numeric columns with mean")
            numeric_cols = df.select_dtypes(include=["number"]).columns
            for col in numeric_cols:
                if df[col].isnull().sum() > 0:
                    mean_val = df[col].mean()
                    df[col] = df[col].fillna(mean_val)
                    print(f"  - Filled '{col}' with mean: {mean_val:.2f}")

        elif strategy == "fill_median":
            print(f"\nStrategy: Fill numeric columns with median")
            numeric_cols = df.select_dtypes(include=["number"]).columns
            for col in numeric_cols:
                if df[col].isnull().sum() > 0:
                    median_val = df[col].median()
                    df[col] = df[col].fillna(median_val)
                    print(f"  - Filled '{col}' with median: {median_val:.2f}")

        elif strategy == "fill_mode":
            print(f"\nStrategy: Fill all columns with mode")
            for col in df.columns:
                if df[col].isnull().sum() > 0:
                    mode_val = df[col].mode()[0] if not df[col].mode().empty else "Unknown"
                    df[col] = df[col].fillna(mode_val)
                    print(f"  - Filled '{col}' with mode: {mode_val}")

        else:
            print(f"[WARNING] Unknown strategy '{strategy}'. Using 'drop'.")
            df = df.dropna()

        missing_after = df.isnull().sum().sum()
        print(f"\nMissing values after cleaning: {missing_after}")
        print(f"Rows: {initial_rows} -> {len(df)}")

        print("=" * 60)
        return df

    except Exception as e:
        print(f"[ERROR] Error handling missing values: {e}")
        return df


def remove_duplicates(df):
    """
    Remove duplicate rows from the dataset.

    This function identifies and removes exact duplicate rows
    to ensure data integrity.

    Parameters:
        df (pandas.DataFrame): The DataFrame to check for duplicates.

    Returns:
        pandas.DataFrame: DataFrame with duplicates removed.

    Examples:
        >>> df = remove_duplicates(df)
        >>> print(f"Removed {initial_rows - len(df)} duplicates")
    """
    try:
        if df is None or df.empty:
            print("[WARNING] DataFrame is None or empty.")
            return df

        print("\n" + "=" * 60)
        print("REMOVING DUPLICATES")
        print("=" * 60)

        initial_rows = len(df)
        duplicate_count = df.duplicated().sum()

        print(f"\nDuplicate rows found: {duplicate_count}")
```

```
        if duplicate_count > 0:
            df = df.drop_duplicates()
            removed = initial_rows - len(df)
            print(f"Duplicate rows removed: {removed}")
            print(f"Remaining rows: {len(df)}")
        else:
            print("No duplicates found. Dataset is clean.")

        print("=" * 60)
        return df

    except Exception as e:
        print(f"[ERROR] Error removing duplicates: {e}")
        return df


def remove_invalid_records(df):
    """
    Remove invalid or inconsistent records from the dataset.

    This function checks for and removes records that contain
    invalid values based on domain-specific rules.

    Parameters:
        df (pandas.DataFrame): The DataFrame to validate.

    Returns:
        pandas.DataFrame: DataFrame with invalid records removed.

    Examples:
        >>> df = remove_invalid_records(df)
    """
    try:
        if df is None or df.empty:
            print("[WARNING] DataFrame is None or empty.")
            return df

        print("\n" + "=" * 60)
        print("REMOVING INVALID RECORDS")
        print("=" * 60)

        initial_rows = len(df)
        invalid_count = 0

        # Check for invalid Age values (must be positive and reasonable)
        if "Age" in df.columns:
            invalid_age = (df["Age"] < 0) | (df["Age"] > 100)
            if invalid_age.any():
                count = invalid_age.sum()
                print(f"\nRemoving {count} rows with invalid Age values (negative or >100)")
                df = df[~invalid_age]
                invalid_count += count

        # Check for invalid Years_of_Experience (must be non-negative)
        if "Years_of_Experience" in df.columns:
            invalid_exp = df["Years_of_Experience"] < 0
            if invalid_exp.any():
                count = invalid_exp.sum()
                print(f"Removing {count} rows with negative Years_of_Experience")
                df = df[~invalid_exp]
                invalid_count += count

        # Check for invalid Routine_Task_Percentage (must be 0-100)
        if "Routine_Task_Percentage" in df.columns:
            invalid_routine = (df["Routine_Task_Percentage"] < 0) | (df["Routine_Task_Percentage"] > 100)
            if invalid_routine.any():
                count = invalid_routine.sum()
                print(f"Removing {count} rows with invalid Routine_Task_Percentage")
                df = df[~invalid_routine]
                invalid_count += count

        # Check for empty string values in categorical columns
        categorical_cols = df.select_dtypes(include=["object"]).columns
        for col in categorical_cols:
```

```
            empty_strings = df[col].astype(str).str.strip() == ""
            if empty_strings.any():
                count = empty_strings.sum()
                print(f"Removing {count} rows with empty '{col}' values")
                df = df[~empty_strings]
                invalid_count += count

        total_removed = initial_rows - len(df)
        print(f"\nTotal invalid records removed: {total_removed}")
        print(f"Remaining valid records: {len(df)}")

        print("=" * 60)
        return df

    except Exception as e:
        print(f"[ERROR] Error removing invalid records: {e}")
        return df


def standardize_column_names(df):
    """
    Standardize column names to a consistent format.

    This function converts column names to lowercase, replaces spaces
    with underscores, and removes special characters for consistency.

    Parameters:
        df (pandas.DataFrame): The DataFrame whose columns to standardize.

    Returns:
        pandas.DataFrame: DataFrame with standardized column names.

    Examples:
        >>> df = standardize_column_names(df)
        >>> print(df.columns.tolist())
        ['age', 'education_level', 'years_of_experience', ...]
    """
    try:
        if df is None or df.empty:
            print("[WARNING] DataFrame is None or empty.")
            return df

        print("\n" + "=" * 60)
        print("STANDARDIZING COLUMN NAMES")
        print("=" * 60)

        original_columns = df.columns.tolist()

        # Standardize column names
        new_columns = []
        for col in original_columns:
            # Convert to lowercase
            new_col = col.lower()
            # Replace spaces with underscores
            new_col = new_col.replace(" ", "_")
            # Remove special characters except underscores
            new_col = re.sub(r"[^a-z0-9_]", "", new_col)
            # Remove multiple consecutive underscores
            new_col = re.sub(r"_+", "_", new_col)
            # Strip leading/trailing underscores
            new_col = new_col.strip("_")
            new_columns.append(new_col)

        # Display changes
        print("\nColumn Name Changes:")
        print("-" * 50)
        for old, new in zip(original_columns, new_columns):
            if old != new:
                print(f"  '{old}' -> '{new}'")

        df.columns = new_columns
        print(f"\nTotal columns standardized: {len(new_columns)}")

        print("=" * 60)
```

```
        return df

    except Exception as e:
        print(f"[ERROR] Error standardizing column names: {e}")
        return df


def save_clean_dataset(df, output_path):
    """
    Save the cleaned DataFrame to a CSV file.

    This function saves the cleaned dataset to the specified output path,
    creating the directory if it doesn't exist.

    Parameters:
        df (pandas.DataFrame): The cleaned DataFrame to save.
        output_path (str): The full path where the CSV file should be saved.

    Returns:
        bool: True if save was successful, False otherwise.

    Examples:
        >>> success = save_clean_dataset(df, "outputs/processed_data/cleaned.csv")
        >>> print(f"Save successful: {success}")
    """
    try:
        if df is None or df.empty:
            print("[WARNING] DataFrame is None or empty. Nothing to save.")
            return False

        # Create output directory if it doesn't exist
        output_dir = os.path.dirname(output_path)
        if not os.path.exists(output_dir):
            os.makedirs(output_dir)
            print(f"\nCreated output directory: {output_dir}")

        # Save the cleaned dataset
        df.to_csv(output_path, index=False)

        print("\n" + "=" * 60)
        print("CLEANED DATASET SAVED")
        print("=" * 60)
        print(f"  Path: {output_path}")
        print(f"  Rows: {len(df)}")
        print(f"  Columns: {len(df.columns)}")
        print(f"  File Size: {os.path.getsize(output_path) / 1024:.2f} KB")
        print("=" * 60)

        return True

    except Exception as e:
        print(f"[ERROR] Error saving cleaned dataset: {e}")
        return False
```

src/exploratory_data_analysis.py

```
"""
Exploratory Data Analysis Module for AI Impact on Jobs & Layoff Risk Analysis.

This module provides functions to perform comprehensive EDA on the dataset.
It includes statistical analysis, distribution analysis, and correlation studies.

Author: Sharma Saurabh (250863131007)
Course: BE05000231 - Python for Data Science (GTU Semester-5)
Month: 2 - Analysis Phase
"""

import pandas as pd
import numpy as np


def basic_statistics(df):
    """
    Display basic statistical summary of the dataset.

    Parameters:
        df (pandas.DataFrame): The DataFrame to analyze.

    Returns:
        pandas.DataFrame: Statistical summary.
    """
    try:
        print("\n" + "=" * 70)
        print("BASIC STATISTICAL SUMMARY")
        print("=" * 70)

        # Numerical columns statistics
        print("\n--- Numerical Columns ---")
        numerical_stats = df.describe()
        print(numerical_stats)

        # Categorical columns statistics
        print("\n--- Categorical Columns ---")
        categorical_cols = df.select_dtypes(include=["object"]).columns
        for col in categorical_cols:
            print(f"\n{col}:")
            print(f"  Unique Values: {df[col].nunique()}")
            print(f"  Top Value: {df[col].mode()[0] if not df[col].mode().empty else 'N/A'}")
            print(f"  Value Counts:")
            for val, count in df[col].value_counts().head(5).items():
                print(f"    {val}: {count} ({count/len(df)*100:.1f}%)")

        print("=" * 70)
        return numerical_stats

    except Exception as e:
        print(f"[ERROR] Error in basic statistics: {e}")
        return None


def distribution_analysis(df):
    """
    Analyze distribution of numerical columns.

    Parameters:
        df (pandas.DataFrame): The DataFrame to analyze.

    Returns:
        dict: Distribution statistics for each numerical column.
    """
    try:
        print("\n" + "=" * 70)
        print("DISTRIBUTION ANALYSIS")
        print("=" * 70)

        numerical_cols = df.select_dtypes(include=["number"]).columns
        distributions = {}
```

```
        for col in numerical_cols:
            skewness = df[col].skew()
            kurtosis = df[col].kurtosis()
            distributions[col] = {
                "mean": df[col].mean(),
                "median": df[col].median(),
                "std": df[col].std(),
                "skewness": skewness,
                "kurtosis": kurtosis,
                "min": df[col].min(),
                "max": df[col].max(),
            }

            print(f"\n{col}:")
            print(f"  Mean: {distributions[col]['mean']:.2f}")
            print(f"  Median: {distributions[col]['median']:.2f}")
            print(f"  Std: {distributions[col]['std']:.2f}")
            print(f"  Skewness: {skewness:.2f}")
            print(f"  Kurtosis: {kurtosis:.2f}")

        print("=" * 70)
        return distributions

    except Exception as e:
        print(f"[ERROR] Error in distribution analysis: {e}")
        return {}


def correlation_analysis(df):
    """
    Compute correlation matrix for numerical columns.

    Parameters:
        df (pandas.DataFrame): The DataFrame to analyze.

    Returns:
        pandas.DataFrame: Correlation matrix.
    """
    try:
        print("\n" + "=" * 70)
        print("CORRELATION ANALYSIS")
        print("=" * 70)

        numerical_cols = df.select_dtypes(include=["number"]).columns
        corr_matrix = df[numerical_cols].corr()

        print("\nCorrelation Matrix:")
        print(corr_matrix.round(2))

        # Find highly correlated pairs
        print("\n--- Highly Correlated Pairs (|r| > 0.5) ---")
        high_corr = []
        for i in range(len(corr_matrix.columns)):
            for j in range(i + 1, len(corr_matrix.columns)):
                if abs(corr_matrix.iloc[i, j]) > 0.5:
                    high_corr.append((
                        corr_matrix.columns[i],
                        corr_matrix.columns[j],
                        corr_matrix.iloc[i, j]
                    ))

        if high_corr:
            for col1, col2, corr in high_corr:
                print(f"  {col1} <-> {col2}: {corr:.2f}")
        else:
            print("  No highly correlated pairs found.")

        print("=" * 70)
        return corr_matrix

    except Exception as e:
        print(f"[ERROR] Error in correlation analysis: {e}")
        return None
```

```
def target_analysis(df, target_col="layoff_risk"):
    """
    Analyze the distribution of the target variable.

    Parameters:
        df (pandas.DataFrame): The DataFrame to analyze.
        target_col (str): Name of the target column.

    Returns:
        pandas.Series: Value counts of target variable.
    """
    try:
        print("\n" + "=" * 70)
        print(f"TARGET VARIABLE ANALYSIS: {target_col}")
        print("=" * 70)

        if target_col not in df.columns:
            print(f"[ERROR] Target column '{target_col}' not found.")
            return None

        target_counts = df[target_col].value_counts()
        target_percent = df[target_col].value_counts(normalize=True) * 100

        print(f"\nTarget Distribution:")
        print("-" * 40)
        for val, count in target_counts.items():
            percent = target_percent[val]
            print(f"  {val}: {count} ({percent:.1f}%)")

        print(f"\nTotal Samples: {len(df)}")
        print(f"Classes: {df[target_col].nunique()}")

        print("=" * 70)
        return target_counts

    except Exception as e:
        print(f"[ERROR] Error in target analysis: {e}")
        return None


def categorical_analysis(df):
    """
    Analyze distribution of categorical columns.

    Parameters:
        df (pandas.DataFrame): The DataFrame to analyze.

    Returns:
        dict: Value counts for each categorical column.
    """
    try:
        print("\n" + "=" * 70)
        print("CATEGORICAL ANALYSIS")
        print("=" * 70)

        categorical_cols = df.select_dtypes(include=["object"]).columns
        cat_analysis = {}

        for col in categorical_cols:
            value_counts = df[col].value_counts()
            cat_analysis[col] = value_counts

            print(f"\n{col} ({df[col].nunique()} unique values):")
            print("-" * 40)
            for val, count in value_counts.head(10).items():
                print(f"  {val}: {count} ({count/len(df)*100:.1f}%)")

        print("=" * 70)
        return cat_analysis

    except Exception as e:
        print(f"[ERROR] Error in categorical analysis: {e}")
```

```
        return {}


def group_analysis(df, group_col, target_col="layoff_risk"):
    """
    Analyze target distribution across groups.

    Parameters:
        df (pandas.DataFrame): The DataFrame to analyze.
        group_col (str): Column to group by.
        target_col (str): Target column for analysis.

    Returns:
        pandas.DataFrame: Grouped analysis results.
    """
    try:
        print("\n" + "=" * 70)
        print(f"GROUP ANALYSIS: {group_col} vs {target_col}")
        print("=" * 70)

        if group_col not in df.columns or target_col not in df.columns:
            print(f"[ERROR] Column not found.")
            return None

        grouped = df.groupby(group_col)[target_col].value_counts().unstack(fill_value=0)
        print(f"\n{grouped}")

        print("=" * 70)
        return grouped

    except Exception as e:
        print(f"[ERROR] Error in group analysis: {e}")
        return None


def run_complete_eda(df):
    """
    Run complete Exploratory Data Analysis.

    Parameters:
        df (pandas.DataFrame): The cleaned DataFrame.

    Returns:
        dict: Complete EDA results.
    """
    try:
        print("\n" + "#" * 70)
        print("# EXPLORATORY DATA ANALYSIS")
        print("#" * 70)

        results = {}

        # Basic Statistics
        results["basic_stats"] = basic_statistics(df)

        # Distribution Analysis
        results["distributions"] = distribution_analysis(df)

        # Correlation Analysis
        results["correlation"] = correlation_analysis(df)

        # Target Analysis
        results["target"] = target_analysis(df)

        # Categorical Analysis
        results["categorical"] = categorical_analysis(df)

        # Group Analysis for key columns
        key_columns = ["industry", "education_level", "company_size", "ai_adoption_level"]
        results["group_analysis"] = {}
        for col in key_columns:
            if col in df.columns:
                results["group_analysis"][col] = group_analysis(df, col)
```

```
        print("\n" + "#" * 70)
        print("# EDA COMPLETED SUCCESSFULLY")
        print("#" * 70)

        return results

    except Exception as e:
        print(f"[ERROR] Error in complete EDA: {e}")
        return {}
```

src/feature_engineering.py

```
"""
Feature Engineering Module for AI Impact on Jobs & Layoff Risk Analysis.

This module provides functions to create new features from existing data.
It enhances the dataset with derived features for better analysis.

Author: Sharma Saurabh (250863131007)
Course: BE05000231 - Python for Data Science (GTU Semester-5)
Month: 2 - Analysis Phase
"""

import pandas as pd
import numpy as np


def create_experience_categories(df):
    """
    Create experience categories from years of experience.

    Parameters:
        df (pandas.DataFrame): The DataFrame to modify.

    Returns:
        pandas.DataFrame: DataFrame with new experience_category column.
    """
    try:
        print("\n[Feature Engineering] Creating experience categories...")

        if "years_of_experience" not in df.columns:
            print("[WARNING] Column 'years_of_experience' not found.")
            return df

        bins = [0, 5, 10, 20, 50]
        labels = ["Junior", "Mid-Level", "Senior", "Expert"]
        df["experience_category"] = pd.cut(
            df["years_of_experience"], bins=bins, labels=labels, include_lowest=True
        )

        print(f"  Created 'experience_category' with {df['experience_category'].nunique()} categories")
        return df

    except Exception as e:
        print(f"[ERROR] Error creating experience categories: {e}")
        return df


def create_age_groups(df):
    """
    Create age groups from age column.

    Parameters:
        df (pandas.DataFrame): The DataFrame to modify.

    Returns:
        pandas.DataFrame: DataFrame with new age_group column.
    """
    try:
        print("\n[Feature Engineering] Creating age groups...")

        if "age" not in df.columns:
            print("[WARNING] Column 'age' not found.")
            return df

        bins = [0, 25, 35, 45, 55, 100]
        labels = ["18-25", "26-35", "36-45", "46-55", "56+"]
        df["age_group"] = pd.cut(
            df["age"], bins=bins, labels=labels, include_lowest=True
        )

        print(f"  Created 'age_group' with {df['age_group'].nunique()} groups")
        return df
```

```
    except Exception as e:
        print(f"[ERROR] Error creating age groups: {e}")
        return df


def create_ai_intensity_score(df):
    """
    Create AI intensity score based on AI-related columns.

    Parameters:
        df (pandas.DataFrame): The DataFrame to modify.

    Returns:
        pandas.DataFrame: DataFrame with new ai_intensity_score column.
    """
    try:
        print("\n[Feature Engineering] Creating AI intensity score...")

        required_cols = ["ai_usage_hours_per_week", "number_of_ai_tools_used", "tasks_automated_percentage"]
        if not all(col in df.columns for col in required_cols):
            print("[WARNING] Required columns not found for AI intensity score.")
            return df

        # Normalize columns to 0-1 scale
        ai_hours_norm = df["ai_usage_hours_per_week"] / df["ai_usage_hours_per_week"].max()
        ai_tools_norm = df["number_of_ai_tools_used"] / df["number_of_ai_tools_used"].max()
        ai_auto_norm = df["tasks_automated_percentage"] / 100

        # Calculate weighted score
        df["ai_intensity_score"] = (
            ai_hours_norm * 0.4 + ai_tools_norm * 0.3 + ai_auto_norm * 0.3
        ) * 100

        print(f"  Created 'ai_intensity_score' (range: {df['ai_intensity_score'].min():.1f} - {df['ai_intensit
y_score'].max():.1f})")
        return df

    except Exception as e:
        print(f"[ERROR] Error creating AI intensity score: {e}")
        return df


def create_routine_creativity_ratio(df):
    """
    Create ratio of routine tasks to creativity requirements.

    Parameters:
        df (pandas.DataFrame): The DataFrame to modify.

    Returns:
        pandas.DataFrame: DataFrame with new routine_creativity_ratio column.
    """
    try:
        print("\n[Feature Engineering] Creating routine-creativity ratio...")

        if "routine_task_percentage" not in df.columns or "creativity_requirement" not in df.columns:
            print("[WARNING] Required columns not found.")
            return df

        # Avoid division by zero
        df["routine_creativity_ratio"] = df["routine_task_percentage"] / (df["creativity_requirement"] + 1)

        print(f"  Created 'routine_creativity_ratio' (range: {df['routine_creativity_ratio'].min():.2f} - {df[
'routine_creativity_ratio'].max():.2f})")
        return df

    except Exception as e:
        print(f"[ERROR] Error creating routine-creativity ratio: {e}")
        return df


def create_human_ai_balance(df):
    """
```

```
    Create balance score between human interaction and AI usage.

    Parameters:
        df (pandas.DataFrame): The DataFrame to modify.

    Returns:
        pandas.DataFrame: DataFrame with new human_ai_balance column.
    """
    try:
        print("\n[Feature Engineering] Creating human-AI balance score...")

        if "human_interaction_level" not in df.columns or "ai_adoption_level" not in df.columns:
            print("[WARNING] Required columns not found.")
            return df

        # Map AI adoption to numeric
        ai_map = {"Low": 1, "Medium": 2, "High": 3}
        ai_numeric = df["ai_adoption_level"].map(ai_map).fillna(1)

        # Normalize human interaction
        human_norm = df["human_interaction_level"] / df["human_interaction_level"].max()

        # Balance score: high human + low AI = high balance
        df["human_ai_balance"] = (human_norm * 50) + ((4 - ai_numeric) / 3 * 50)

        print(f"  Created 'human_ai_balance' (range: {df['human_ai_balance'].min():.1f} - {df['human_ai_balanc
e'].max():.1f})")
        return df

    except Exception as e:
        print(f"[ERROR] Error creating human-AI balance: {e}")
        return df


def create_risk_factors(df):
    """
    Create risk factor flags based on domain knowledge.

    Parameters:
        df (pandas.DataFrame): The DataFrame to modify.

    Returns:
        pandas.DataFrame: DataFrame with new risk factor columns.
    """
    try:
        print("\n[Feature Engineering] Creating risk factor flags...")

        # High automation risk
        if "tasks_automated_percentage" in df.columns:
            df["high_automation_risk"] = (df["tasks_automated_percentage"] > 70).astype(int)
            print(f"  Created 'high_automation_risk' ({df['high_automation_risk'].sum()} flagged)")

        # Low creativity risk
        if "creativity_requirement" in df.columns:
            df["low_creativity_risk"] = (df["creativity_requirement"] < 30).astype(int)
            print(f"  Created 'low_creativity_risk' ({df['low_creativity_risk'].sum()} flagged)")

        # High routine risk
        if "routine_task_percentage" in df.columns:
            df["high_routine_risk"] = (df["routine_task_percentage"] > 70).astype(int)
            print(f"  Created 'high_routine_risk' ({df['high_routine_risk'].sum()} flagged)")

        # Low human interaction risk
        if "human_interaction_level" in df.columns:
            df["low_interaction_risk"] = (df["human_interaction_level"] < 30).astype(int)
            print(f"  Created 'low_interaction_risk' ({df['low_interaction_risk'].sum()} flagged)")

        return df

    except Exception as e:
        print(f"[ERROR] Error creating risk factors: {e}")
        return df
```

```
def run_feature_engineering(df):
    """
    Run complete feature engineering pipeline.

    Parameters:
        df (pandas.DataFrame): The cleaned DataFrame.

    Returns:
        pandas.DataFrame: DataFrame with engineered features.
    """
    try:
        print("\n" + "#" * 70)
        print("# FEATURE ENGINEERING")
        print("#" * 70)

        initial_cols = len(df.columns)

        # Apply all feature engineering functions
        df = create_experience_categories(df)
        df = create_age_groups(df)
        df = create_ai_intensity_score(df)
        df = create_routine_creativity_ratio(df)
        df = create_human_ai_balance(df)
        df = create_risk_factors(df)

        final_cols = len(df.columns)
        print(f"\nFeatures added: {final_cols - initial_cols}")
        print(f"Total columns: {final_cols}")

        print("\n" + "#" * 70)
        print("# FEATURE ENGINEERING COMPLETED SUCCESSFULLY")
        print("#" * 70)

        return df

    except Exception as e:
        print(f"[ERROR] Error in feature engineering: {e}")
        return df
```

src/visualization.py

```
"""
Visualization Module for AI Impact on Jobs & Layoff Risk Analysis.

This module provides functions to create various charts and plots
for data exploration and presentation.

Author: Sharma Saurabh (250863131007)
Course: BE05000231 - Python for Data Science (GTU Semester-5)
Month: 2 - Analysis Phase
"""

import pandas as pd
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import seaborn as sns
import os


def save_plot(fig, filename, output_dir):
    """
    Save plot to specified directory.

    Parameters:
        fig (matplotlib.figure.Figure): The figure to save.
        filename (str): Name of the output file.
        output_dir (str): Directory to save the file.
    """
    try:
        if not os.path.exists(output_dir):
            os.makedirs(output_dir)

        filepath = os.path.join(output_dir, filename)
        fig.savefig(filepath, dpi=150, bbox_inches="tight")
        plt.close(fig)
        print(f"  Saved: {filename}")

    except Exception as e:
        print(f"[ERROR] Error saving plot: {e}")


def plot_target_distribution(df, target_col="layoff_risk", output_dir="outputs/charts"):
    """
    Plot distribution of target variable.

    Parameters:
        df (pandas.DataFrame): The DataFrame to plot.
        target_col (str): Target column name.
        output_dir (str): Output directory for plots.
    """
    try:
        print("\n[Visualization] Creating target distribution plot...")

        fig, axes = plt.subplots(1, 2, figsize=(12, 5))

        # Bar chart
        target_counts = df[target_col].value_counts()
        colors = ["#2ecc71", "#f39c12", "#e74c3c"]
        axes[0].bar(target_counts.index, target_counts.values, color=colors[:len(target_counts)])
        axes[0].set_title("Layoff Risk Distribution", fontsize=14, fontweight="bold")
        axes[0].set_xlabel("Risk Level")
        axes[0].set_ylabel("Count")
        for i, v in enumerate(target_counts.values):
            axes[0].text(i, v + 50, str(v), ha="center", fontweight="bold")

        # Pie chart
        axes[1].pie(target_counts.values, labels=target_counts.index, autopct="%1.1f%%",
                    colors=colors[:len(target_counts)], startangle=90)
        axes[1].set_title("Layoff Risk Percentage", fontsize=14, fontweight="bold")
```

```
        plt.tight_layout()
        save_plot(fig, "target_distribution.png", output_dir)

    except Exception as e:
        print(f"[ERROR] Error plotting target distribution: {e}")


def plot_numerical_distributions(df, output_dir="outputs/charts"):
    """
    Plot distributions of numerical columns.

    Parameters:
        df (pandas.DataFrame): The DataFrame to plot.
        output_dir (str): Output directory for plots.
    """
    try:
        print("\n[Visualization] Creating numerical distribution plots...")

        numerical_cols = ["age", "years_of_experience", "routine_task_percentage",
                          "creativity_requirement", "human_interaction_level",
                          "number_of_ai_tools_used", "ai_usage_hours_per_week",
                          "tasks_automated_percentage", "ai_training_hours"]

        available_cols = [col for col in numerical_cols if col in df.columns]

        fig, axes = plt.subplots(3, 3, figsize=(15, 15))
        axes = axes.flatten()

        for i, col in enumerate(available_cols[:9]):
            axes[i].hist(df[col], bins=30, color="#3498db", edgecolor="black", alpha=0.7)
            axes[i].set_title(f"{col.replace('_', ' ').title()}", fontsize=12, fontweight="bold")
            axes[i].set_xlabel(col.replace('_', ' ').title())
            axes[i].set_ylabel("Frequency")
            axes[i].axvline(df[col].mean(), color="red", linestyle="--", label=f"Mean: {df[col].mean():.1f}")
            axes[i].legend()

        # Hide unused subplots
        for j in range(len(available_cols), 9):
            axes[j].set_visible(False)

        plt.suptitle("Numerical Column Distributions", fontsize=16, fontweight="bold", y=1.02)
        plt.tight_layout()
        save_plot(fig, "numerical_distributions.png", output_dir)

    except Exception as e:
        print(f"[ERROR] Error plotting numerical distributions: {e}")


def plot_categorical_distributions(df, output_dir="outputs/charts"):
    """
    Plot distributions of categorical columns.

    Parameters:
        df (pandas.DataFrame): The DataFrame to plot.
        output_dir (str): Output directory for plots.
    """
    try:
        print("\n[Visualization] Creating categorical distribution plots...")

        categorical_cols = ["education_level", "industry", "company_size",
                            "job_level", "ai_adoption_level"]

        available_cols = [col for col in categorical_cols if col in df.columns]

        fig, axes = plt.subplots(2, 3, figsize=(18, 12))
        axes = axes.flatten()

        colors = sns.color_palette("Set2", len(available_cols))

        for i, col in enumerate(available_cols[:6]):
            value_counts = df[col].value_counts()
            axes[i].barh(value_counts.index, value_counts.values, color=colors[i])
            axes[i].set_title(f"{col.replace('_', ' ').title()}", fontsize=12, fontweight="bold")
            axes[i].set_xlabel("Count")
```

```
            for j, v in enumerate(value_counts.values):
                axes[i].text(v + 20, j, str(v), va="center", fontweight="bold")

        # Hide unused subplots
        for j in range(len(available_cols), 6):
            axes[j].set_visible(False)

        plt.suptitle("Categorical Column Distributions", fontsize=16, fontweight="bold", y=1.02)
        plt.tight_layout()
        save_plot(fig, "categorical_distributions.png", output_dir)

    except Exception as e:
        print(f"[ERROR] Error plotting categorical distributions: {e}")


def plot_correlation_heatmap(df, output_dir="outputs/charts"):
    """
    Plot correlation heatmap for numerical columns.

    Parameters:
        df (pandas.DataFrame): The DataFrame to plot.
        output_dir (str): Output directory for plots.
    """
    try:
        print("\n[Visualization] Creating correlation heatmap...")

        numerical_cols = df.select_dtypes(include=["number"]).columns
        corr_matrix = df[numerical_cols].corr()

        fig, ax = plt.subplots(figsize=(14, 10))
        mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
        sns.heatmap(corr_matrix, mask=mask, annot=True, fmt=".2f", cmap="coolwarm",
                    center=0, square=True, linewidths=0.5, ax=ax)
        ax.set_title("Correlation Heatmap", fontsize=16, fontweight="bold")
        plt.tight_layout()
        save_plot(fig, "correlation_heatmap.png", output_dir)

    except Exception as e:
        print(f"[ERROR] Error plotting correlation heatmap: {e}")


def plot_risk_by_category(df, category_col, target_col="layoff_risk", output_dir="outputs/charts"):
    """
    Plot layoff risk distribution by category.

    Parameters:
        df (pandas.DataFrame): The DataFrame to plot.
        category_col (str): Category column to analyze.
        target_col (str): Target column name.
        output_dir (str): Output directory for plots.
    """
    try:
        print(f"\n[Visualization] Creating risk by {category_col} plot...")

        if category_col not in df.columns or target_col not in df.columns:
            print(f"[WARNING] Required columns not found.")
            return

        fig, ax = plt.subplots(figsize=(12, 6))

        ct = pd.crosstab(df[category_col], df[target_col], normalize="index") * 100
        ct.plot(kind="bar", stacked=True, ax=ax, color=["#2ecc71", "#f39c12", "#e74c3c"])

        ax.set_title(f"Layoff Risk by {category_col.replace('_', ' ').title()}",
                     fontsize=14, fontweight="bold")
        ax.set_xlabel(category_col.replace("_", " ").title())
        ax.set_ylabel("Percentage")
        ax.legend(title="Risk Level")
        plt.xticks(rotation=45, ha="right")
        plt.tight_layout()
        save_plot(fig, f"risk_by_{category_col}.png", output_dir)

    except Exception as e:
        print(f"[ERROR] Error plotting risk by category: {e}")
```

```
def plot_boxplots(df, output_dir="outputs/charts"):
    """
    Plot boxplots for numerical columns by target.

    Parameters:
        df (pandas.DataFrame): The DataFrame to plot.
        output_dir (str): Output directory for plots.
    """
    try:
        print("\n[Visualization] Creating boxplots...")

        numerical_cols = ["age", "years_of_experience", "routine_task_percentage",
                          "tasks_automated_percentage"]
        target_col = "layoff_risk"

        available_cols = [col for col in numerical_cols if col in df.columns]

        fig, axes = plt.subplots(2, 2, figsize=(14, 10))
        axes = axes.flatten()

        colors = {"Low": "#2ecc71", "Medium": "#f39c12", "High": "#e74c3c"}

        for i, col in enumerate(available_cols[:4]):
            sns.boxplot(data=df, x=target_col, y=col, ax=axes[i],
                        palette=colors, order=["Low", "Medium", "High"])
            axes[i].set_title(f"{col.replace('_', ' ').title()} by Risk Level",
                              fontsize=12, fontweight="bold")
            axes[i].set_xlabel("Layoff Risk")
            axes[i].set_ylabel(col.replace("_", " ").title())

        plt.suptitle("Boxplots: Numerical Features by Risk Level", fontsize=16, fontweight="bold", y=1.02)
        plt.tight_layout()
        save_plot(fig, "boxplots_by_risk.png", output_dir)

    except Exception as e:
        print(f"[ERROR] Error plotting boxplots: {e}")


def plot_ai_adoption_analysis(df, output_dir="outputs/charts"):
    """
    Plot AI adoption analysis.

    Parameters:
        df (pandas.DataFrame): The DataFrame to plot.
        output_dir (str): Output directory for plots.
    """
    try:
        print("\n[Visualization] Creating AI adoption analysis plot...")

        fig, axes = plt.subplots(1, 2, figsize=(14, 6))

        # AI Adoption Level Distribution
        if "ai_adoption_level" in df.columns:
            ai_counts = df["ai_adoption_level"].value_counts()
            colors = ["#3498db", "#f39c12", "#e74c3c"]
            axes[0].bar(ai_counts.index, ai_counts.values, color=colors[:len(ai_counts)])
            axes[0].set_title("AI Adoption Level Distribution", fontsize=14, fontweight="bold")
            axes[0].set_xlabel("AI Adoption Level")
            axes[0].set_ylabel("Count")

        # AI Usage vs Tasks Automated
        if "ai_usage_hours_per_week" in df.columns and "tasks_automated_percentage" in df.columns:
            scatter = axes[1].scatter(df["ai_usage_hours_per_week"],
                                      df["tasks_automated_percentage"],
                                      c=df["layoff_risk"].map({"Low": 0, "Medium": 1, "High": 2}),
                                      cmap="RdYlGn_r", alpha=0.5)
            axes[1].set_title("AI Usage vs Tasks Automated", fontsize=14, fontweight="bold")
            axes[1].set_xlabel("AI Usage Hours/Week")
            axes[1].set_ylabel("Tasks Automated %")

        plt.tight_layout()
        save_plot(fig, "ai_adoption_analysis.png", output_dir)
```

```
    except Exception as e:
        print(f"[ERROR] Error plotting AI adoption analysis: {e}")


def plot_feature_importance_preview(df, target_col="layoff_risk", output_dir="outputs/charts"):
    """
    Plot feature importance preview using correlation with target.

    Parameters:
        df (pandas.DataFrame): The DataFrame to plot.
        target_col (str): Target column name.
        output_dir (str): Output directory for plots.
    """
    try:
        print("\n[Visualization] Creating feature importance preview...")

        numerical_cols = df.select_dtypes(include=["number"]).columns
        if len(numerical_cols) == 0:
            print("[WARNING] No numerical columns found.")
            return

        # Calculate correlation with target (encode target first)
        target_encoded = df[target_col].map({"Low": 0, "Medium": 1, "High": 2})

        correlations = {}
        for col in numerical_cols:
            correlations[col] = abs(df[col].corr(target_encoded))

        corr_series = pd.Series(correlations).sort_values(ascending=True)

        fig, ax = plt.subplots(figsize=(10, 8))
        corr_series.plot(kind="barh", ax=ax, color="#3498db")
        ax.set_title("Feature Correlation with Target (Absolute)", fontsize=14, fontweight="bold")
        ax.set_xlabel("Absolute Correlation")
        ax.set_ylabel("Features")
        plt.tight_layout()
        save_plot(fig, "feature_importance_preview.png", output_dir)

    except Exception as e:
        print(f"[ERROR] Error plotting feature importance: {e}")


def run_complete_visualization(df, output_dir="outputs/charts"):
    """
    Run complete visualization pipeline.

    Parameters:
        df (pandas.DataFrame): The engineered DataFrame.
        output_dir (str): Output directory for all plots.

    Returns:
        list: List of generated plot filenames.
    """
    try:
        print("\n" + "#" * 70)
        print("# DATA VISUALIZATION")
        print("#" * 70)

        # Generate all plots
        plot_target_distribution(df, output_dir=output_dir)
        plot_numerical_distributions(df, output_dir=output_dir)
        plot_categorical_distributions(df, output_dir=output_dir)
        plot_correlation_heatmap(df, output_dir=output_dir)

        # Risk by category plots
        for col in ["industry", "education_level", "company_size", "ai_adoption_level"]:
            if col in df.columns:
                plot_risk_by_category(df, col, output_dir=output_dir)

        plot_boxplots(df, output_dir=output_dir)
        plot_ai_adoption_analysis(df, output_dir=output_dir)
        plot_feature_importance_preview(df, output_dir=output_dir)
```

```
        print("\n" + "#" * 70)
        print("# VISUALIZATION COMPLETED SUCCESSFULLY")
        print("#" * 70)

    except Exception as e:
        print(f"[ERROR] Error in visualization: {e}")
```

src/preprocessing.py

```
"""
Preprocessing Module for AI Impact on Jobs & Layoff Risk Analysis.

This module provides functions to preprocess data for machine learning.
It includes encoding, scaling, and train-test splitting.

Author: Sharma Saurabh (250863131007)
Course: BE05000231 - Python for Data Science (GTU Semester-5)
Month: 2 - Analysis Phase
"""

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
import joblib
import os


def encode_categorical_columns(df, target_col="layoff_risk"):
    """
    Encode categorical columns using Label Encoding.

    Parameters:
        df (pandas.DataFrame): The DataFrame to encode.
        target_col (str): Target column name.

    Returns:
        pandas.DataFrame: DataFrame with encoded columns.
        dict: Mapping of encoders for each column.
    """
    try:
        print("\n[Preprocessing] Encoding categorical columns...")

        categorical_cols = df.select_dtypes(include=["object", "category"]).columns
        encoders = {}

        df_encoded = df.copy()

        for col in categorical_cols:
            if col == target_col:
                continue

            le = LabelEncoder()
            df_encoded[col] = le.fit_transform(df_encoded[col].astype(str))
            encoders[col] = le
            print(f"  Encoded '{col}': {len(le.classes_)} classes")

        # Encode target column
        if target_col in df_encoded.columns:
            le_target = LabelEncoder()
            df_encoded[target_col] = le_target.fit_transform(df_encoded[target_col])
            encoders[target_col] = le_target
            print(f"  Encoded target '{target_col}': {list(le_target.classes_)}")

        print(f"\nTotal columns encoded: {len(encoders)}")
        return df_encoded, encoders

    except Exception as e:
        print(f"[ERROR] Error encoding categorical columns: {e}")
        return df, {}


def scale_numerical_columns(df, exclude_cols=None):
    """
    Scale numerical columns using StandardScaler.

    Parameters:
        df (pandas.DataFrame): The DataFrame to scale.
        exclude_cols (list): Columns to exclude from scaling.
```

```
    Returns:
        pandas.DataFrame: DataFrame with scaled columns.
        StandardScaler: The fitted scaler object.
    """
    try:
        print("\n[Preprocessing] Scaling numerical columns...")

        if exclude_cols is None:
            exclude_cols = []

        numerical_cols = df.select_dtypes(include=["number"]).columns
        cols_to_scale = [col for col in numerical_cols if col not in exclude_cols]

        if len(cols_to_scale) == 0:
            print("[WARNING] No numerical columns to scale.")
            return df, None

        scaler = StandardScaler()
        df_scaled = df.copy()
        df_scaled[cols_to_scale] = scaler.fit_transform(df[cols_to_scale])

        print(f"  Scaled {len(cols_to_scale)} numerical columns")
        print(f"  Columns: {', '.join(cols_to_scale)}")

        return df_scaled, scaler

    except Exception as e:
        print(f"[ERROR] Error scaling numerical columns: {e}")
        return df, None


def split_features_target(df, target_col="layoff_risk"):
    """
    Split DataFrame into features (X) and target (y).

    Parameters:
        df (pandas.DataFrame): The DataFrame to split.
        target_col (str): Target column name.

    Returns:
        pandas.DataFrame: Features DataFrame.
        pandas.Series: Target Series.
    """
    try:
        print("\n[Preprocessing] Splitting features and target...")

        if target_col not in df.columns:
            print(f"[ERROR] Target column '{target_col}' not found.")
            return None, None

        X = df.drop(columns=[target_col])
        y = df[target_col]

        print(f"  Features shape: {X.shape}")
        print(f"  Target shape: {y.shape}")
        print(f"  Target distribution:\n{y.value_counts()}")

        return X, y

    except Exception as e:
        print(f"[ERROR] Error splitting features and target: {e}")
        return None, None


def train_test_split_data(X, y, test_size=0.2, random_state=42):
    """
    Split data into training and testing sets.

    Parameters:
        X (pandas.DataFrame): Features.
        y (pandas.Series): Target.
        test_size (float): Proportion of data for testing.
        random_state (int): Random seed for reproducibility.
```

```
    Returns:
        tuple: (X_train, X_test, y_train, y_test)
    """
    try:
        print("\n[Preprocessing] Splitting train and test data...")

        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=test_size, random_state=random_state, stratify=y
        )

        print(f"  Training set: {X_train.shape[0]} samples")
        print(f"  Testing set: {X_test.shape[0]} samples")
        print(f"  Train target distribution:\n{y_train.value_counts()}")
        print(f"  Test target distribution:\n{y_test.value_counts()}")

        return X_train, X_test, y_train, y_test

    except Exception as e:
        print(f"[ERROR] Error in train-test split: {e}")
        return None, None, None, None


def save_preprocessor(scaler, encoders, output_dir):
    """
    Save preprocessor objects to disk.

    Parameters:
        scaler (StandardScaler): The fitted scaler.
        encoders (dict): Dictionary of fitted encoders.
        output_dir (str): Output directory path.
    """
    try:
        print("\n[Preprocessing] Saving preprocessor objects...")

        if not os.path.exists(output_dir):
            os.makedirs(output_dir)

        if scaler is not None:
            scaler_path = os.path.join(output_dir, "scaler.pkl")
            joblib.dump(scaler, scaler_path)
            print(f"  Saved scaler: {scaler_path}")

        if encoders:
            encoder_path = os.path.join(output_dir, "encoder.pkl")
            joblib.dump(encoders, encoder_path)
            print(f"  Saved encoders: {encoder_path}")

    except Exception as e:
        print(f"[ERROR] Error saving preprocessor: {e}")


def run_preprocessing(df, config):
    """
    Run complete preprocessing pipeline.

    Parameters:
        df (pandas.DataFrame): The engineered DataFrame.
        config: Configuration module.

    Returns:
        tuple: (X_train, X_test, y_train, y_test, scaler, encoders)
    """
    try:
        print("\n" + "#" * 70)
        print("# DATA PREPROCESSING")
        print("#" * 70)

        # Step 1: Encode categorical columns
        df_encoded, encoders = encode_categorical_columns(df, config.TARGET_COLUMN)

        # Step 2: Split features and target
        X, y = split_features_target(df_encoded, config.TARGET_COLUMN)

        if X is None or y is None:
```

```
            print("[ERROR] Failed to split features and target.")
            return None, None, None, None, None, None

        # Step 3: Scale numerical columns
        X_scaled, scaler = scale_numerical_columns(X)

        # Step 4: Train-test split
        X_train, X_test, y_train, y_test = train_test_split_data(
            X_scaled, y, test_size=config.TEST_SIZE, random_state=config.RANDOM_STATE
        )

        # Step 5: Save preprocessor objects
        save_preprocessor(scaler, encoders, config.MODELS_OUTPUT_DIR)

        # Step 6: Save processed data
        train_data = pd.concat([X_train, y_train], axis=1)
        test_data = pd.concat([X_test, y_test], axis=1)

        train_data.to_csv(config.TRAIN_DATA_PATH, index=False)
        test_data.to_csv(config.TEST_DATA_PATH, index=False)
        print(f"\nSaved training data: {config.TRAIN_DATA_PATH}")
        print(f"Saved testing data: {config.TEST_DATA_PATH}")

        print("\n" + "#" * 70)
        print("# PREPROCESSING COMPLETED SUCCESSFULLY")
        print("#" * 70)

        return X_train, X_test, y_train, y_test, scaler, encoders

    except Exception as e:
        print(f"[ERROR] Error in preprocessing: {e}")
        return None, None, None, None, None, None
```

src/model_training.py

```
"""
Model Training Module for AI Impact on Jobs & Layoff Risk Analysis.

This module provides functions to train multiple machine learning models.
It includes model selection, training, and comparison.

Author: Sharma Saurabh (250863131007)
Course: BE05000231 - Python for Data Science (GTU Semester-5)
Month: 3 - Modeling Phase
"""

import pandas as pd
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
import joblib
import os
import time


def initialize_models():
    """
    Initialize multiple machine learning models for comparison.

    Returns:
        dict: Dictionary of initialized models.
    """
    try:
        print("\n[Model Training] Initializing models...")

        models = {
            "Logistic Regression": LogisticRegression(
                max_iter=1000,
                random_state=42
            ),
            "Decision Tree": DecisionTreeClassifier(
                max_depth=10,
                random_state=42
            ),
            "Random Forest": RandomForestClassifier(
                n_estimators=100,
                max_depth=10,
                random_state=42,
                n_jobs=-1
            )
        }

        for name, model in models.items():
            print(f"  - {name}: Initialized")

        return models

    except Exception as e:
        print(f"[ERROR] Error initializing models: {e}")
        return {}


def train_model(model, X_train, y_train, model_name="Model"):
    """
    Train a single machine learning model.

    Parameters:
        model: The model to train.
        X_train (pandas.DataFrame): Training features.
        y_train (pandas.Series): Training target.
        model_name (str): Name of the model for display.

    Returns:
        tuple: (trained_model, training_time)
```

```
    """
    try:
        print(f"\n  Training {model_name}...")

        start_time = time.time()
        model.fit(X_train, y_train)
        end_time = time.time()

        training_time = end_time - start_time
        print(f"  {model_name} trained in {training_time:.2f} seconds")

        return model, training_time

    except Exception as e:
        print(f"[ERROR] Error training {model_name}: {e}")
        return None, 0


def train_all_models(models, X_train, y_train):
    """
    Train all initialized models.

    Parameters:
        models (dict): Dictionary of models to train.
        X_train (pandas.DataFrame): Training features.
        y_train (pandas.Series): Training target.

    Returns:
        dict: Dictionary of trained models with metadata.
    """
    try:
        print("\n" + "=" * 70)
        print("TRAINING ALL MODELS")
        print("=" * 70)

        trained_models = {}

        for name, model in models.items():
            trained_model, train_time = train_model(model, X_train, y_train, name)

            if trained_model is not None:
                # Calculate training accuracy
                train_pred = trained_model.predict(X_train)
                train_accuracy = accuracy_score(y_train, train_pred)

                trained_models[name] = {
                    "model": trained_model,
                    "training_time": train_time,
                    "train_accuracy": train_accuracy
                }

                print(f"  {name} - Training Accuracy: {train_accuracy:.4f}")

        print("\n" + "=" * 70)
        print(f"Successfully trained: {len(trained_models)}/{len(models)} models")
        print("=" * 70)

        return trained_models

    except Exception as e:
        print(f"[ERROR] Error training all models: {e}")
        return {}


def save_best_model(trained_models, X_test, y_test, output_dir):
    """
    Save the best performing model to disk.

    Parameters:
        trained_models (dict): Dictionary of trained models.
        X_test (pandas.DataFrame): Testing features.
        y_test (pandas.Series): Testing target.
        output_dir (str): Output directory path.
```

```
    Returns:
        str: Name of the best model.
    """
    try:
        print("\n[Model Training] Finding and saving best model...")

        best_accuracy = 0
        best_model_name = None

        for name, metadata in trained_models.items():
            model = metadata["model"]
            y_pred = model.predict(X_test)
            accuracy = accuracy_score(y_test, y_pred)

            if accuracy > best_accuracy:
                best_accuracy = accuracy
                best_model_name = name

        if best_model_name:
            best_model = trained_models[best_model_name]["model"]

            if not os.path.exists(output_dir):
                os.makedirs(output_dir)

            model_path = os.path.join(output_dir, "best_model.pkl")
            joblib.dump(best_model, model_path)

            print(f"\n  Best Model: {best_model_name}")
            print(f"  Test Accuracy: {best_accuracy:.4f}")
            print(f"  Saved to: {model_path}")

            return best_model_name

        return None

    except Exception as e:
        print(f"[ERROR] Error saving best model: {e}")
        return None


def compare_models(trained_models, X_test, y_test):
    """
    Compare performance of all trained models.

    Parameters:
        trained_models (dict): Dictionary of trained models.
        X_test (pandas.DataFrame): Testing features.
        y_test (pandas.Series): Testing target.

    Returns:
        pandas.DataFrame: Comparison results.
    """
    try:
        print("\n" + "=" * 70)
        print("MODEL COMPARISON")
        print("=" * 70)

        results = []

        for name, metadata in trained_models.items():
            model = metadata["model"]
            y_pred = model.predict(X_test)

            accuracy = accuracy_score(y_test, y_pred)

            results.append({
                "Model": name,
                "Training Time (s)": f"{metadata['training_time']:.2f}",
                "Train Accuracy": f"{metadata['train_accuracy']:.4f}",
                "Test Accuracy": f"{accuracy:.4f}"
            })

        comparison_df = pd.DataFrame(results)
        print("\n" + comparison_df.to_string(index=False))
```

```
        print("\n" + "=" * 70)

        return comparison_df

    except Exception as e:
        print(f"[ERROR] Error comparing models: {e}")
        return None


def run_model_training(X_train, X_test, y_train, y_test, config):
    """
    Run complete model training pipeline.

    Parameters:
        X_train (pandas.DataFrame): Training features.
        X_test (pandas.DataFrame): Testing features.
        y_train (pandas.Series): Training target.
        y_test (pandas.Series): Testing target.
        config: Configuration module.

    Returns:
        tuple: (trained_models, comparison_df, best_model_name)
    """
    try:
        print("\n" + "#" * 70)
        print("# MODEL TRAINING")
        print("#" * 70)

        # Step 1: Initialize models
        models = initialize_models()

        # Step 2: Train all models
        trained_models = train_all_models(models, X_train, y_train)

        # Step 3: Compare models
        comparison_df = compare_models(trained_models, X_test, y_test)

        # Step 4: Save best model
        best_model_name = save_best_model(trained_models, X_test, y_test, config.MODELS_OUTPUT_DIR)

        print("\n" + "#" * 70)
        print("# MODEL TRAINING COMPLETED SUCCESSFULLY")
        print("#" * 70)

        return trained_models, comparison_df, best_model_name

    except Exception as e:
        print(f"[ERROR] Error in model training: {e}")
        return {}, None, None
```

src/model_evaluation.py

```
"""
Model Evaluation Module for AI Impact on Jobs & Layoff Risk Analysis.

This module provides functions to evaluate machine learning models.
It includes accuracy, precision, recall, F1-score, confusion matrix, and ROC curve.

Author: Sharma Saurabh (250863131007)
Course: BE05000231 - Python for Data Science (GTU Semester-5)
Month: 3 - Modeling Phase
"""

import pandas as pd
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_curve,
    auc
)
from sklearn.preprocessing import label_binarize
import os


def calculate_metrics(y_true, y_pred, average="weighted"):
    """
    Calculate classification metrics.

    Parameters:
        y_true (pandas.Series): True labels.
        y_pred (pandas.Series): Predicted labels.
        average (str): Averaging method.

    Returns:
        dict: Dictionary of metrics.
    """
    try:
        metrics = {
            "accuracy": accuracy_score(y_true, y_pred),
            "precision": precision_score(y_true, y_pred, average=average, zero_division=0),
            "recall": recall_score(y_true, y_pred, average=average, zero_division=0),
            "f1_score": f1_score(y_true, y_pred, average=average, zero_division=0)
        }
        return metrics

    except Exception as e:
        print(f"[ERROR] Error calculating metrics: {e}")
        return {}


def print_classification_report(y_true, y_pred, model_name="Model"):
    """
    Print detailed classification report.

    Parameters:
        y_true (pandas.Series): True labels.
        y_pred (pandas.Series): Predicted labels.
        model_name (str): Name of the model.

    Returns:
        str: Classification report string.
    """
    try:
        print(f"\n--- {model_name} Classification Report ---")
```

```
        report = classification_report(y_true, y_pred, zero_division=0)
        print(report)
        return report

    except Exception as e:
        print(f"[ERROR] Error generating classification report: {e}")
        return ""


def plot_confusion_matrix(y_true, y_pred, model_name="Model", output_dir="outputs/charts"):
    """
    Plot confusion matrix heatmap.

    Parameters:
        y_true (pandas.Series): True labels.
        y_pred (pandas.Series): Predicted labels.
        model_name (str): Name of the model.
        output_dir (str): Output directory for plots.
    """
    try:
        print(f"\n[Evaluation] Creating confusion matrix for {model_name}...")

        cm = confusion_matrix(y_true, y_pred)

        fig, ax = plt.subplots(figsize=(8, 6))
        sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", ax=ax,
                    xticklabels=["Low", "Medium", "High"],
                    yticklabels=["Low", "Medium", "High"])
        ax.set_title(f"Confusion Matrix - {model_name}", fontsize=14, fontweight="bold")
        ax.set_xlabel("Predicted Label")
        ax.set_ylabel("True Label")
        plt.tight_layout()

        if not os.path.exists(output_dir):
            os.makedirs(output_dir)

        filename = f"confusion_matrix_{model_name.lower().replace(' ', '_')}.png"
        filepath = os.path.join(output_dir, filename)
        fig.savefig(filepath, dpi=150, bbox_inches="tight")
        plt.close(fig)
        print(f"  Saved: {filename}")

    except Exception as e:
        print(f"[ERROR] Error plotting confusion matrix: {e}")


def plot_roc_curve(model, X_test, y_test, model_name="Model", output_dir="outputs/charts"):
    """
    Plot ROC curve for multi-class classification.

    Parameters:
        model: Trained model.
        X_test (pandas.DataFrame): Testing features.
        y_test (pandas.Series): Testing labels.
        model_name (str): Name of the model.
        output_dir (str): Output directory for plots.
    """
    try:
        print(f"\n[Evaluation] Creating ROC curve for {model_name}...")

        # Binarize the output for multi-class
        classes = sorted(y_test.unique())
        n_classes = len(classes)

        if n_classes < 2:
            print("  Skipping ROC curve (less than 2 classes)")
            return

        y_test_bin = label_binarize(y_test, classes=classes)

        # Get probability predictions
        if hasattr(model, "predict_proba"):
            y_score = model.predict_proba(X_test)
        else:
```

```
            y_score = model.decision_function(X_test)

        # Calculate ROC curve and AUC for each class
        fpr = {}
        tpr = {}
        roc_auc = {}

        for i in range(n_classes):
            fpr[i], tpr[i], _ = roc_curve(y_test_bin[:, i], y_score[:, i])
            roc_auc[i] = auc(fpr[i], tpr[i])

        # Plot ROC curves
        fig, ax = plt.subplots(figsize=(8, 6))
        colors = ["#2ecc71", "#f39c12", "#e74c3c"]

        for i, color in zip(range(n_classes), colors[:n_classes]):
            ax.plot(fpr[i], tpr[i], color=color, linewidth=2,
                    label=f"Class {classes[i]} (AUC = {roc_auc[i]:.2f})")

        ax.plot([0, 1], [0, 1], "k--", linewidth=1)
        ax.set_xlim([0.0, 1.0])
        ax.set_ylim([0.0, 1.05])
        ax.set_xlabel("False Positive Rate", fontsize=12)
        ax.set_ylabel("True Positive Rate", fontsize=12)
        ax.set_title(f"ROC Curve - {model_name}", fontsize=14, fontweight="bold")
        ax.legend(loc="lower right")
        plt.tight_layout()

        if not os.path.exists(output_dir):
            os.makedirs(output_dir)

        filename = f"roc_curve_{model_name.lower().replace(' ', '_')}.png"
        filepath = os.path.join(output_dir, filename)
        fig.savefig(filepath, dpi=150, bbox_inches="tight")
        plt.close(fig)
        print(f"  Saved: {filename}")

    except Exception as e:
        print(f"[ERROR] Error plotting ROC curve: {e}")


def evaluate_model(model, X_test, y_test, model_name="Model", output_dir="outputs/charts"):
    """
    Complete evaluation of a single model.

    Parameters:
        model: Trained model.
        X_test (pandas.DataFrame): Testing features.
        y_test (pandas.Series): Testing labels.
        model_name (str): Name of the model.
        output_dir (str): Output directory for plots.

    Returns:
        dict: Evaluation metrics.
    """
    try:
        print("\n" + "=" * 70)
        print(f"EVALUATING: {model_name}")
        print("=" * 70)

        # Make predictions
        y_pred = model.predict(X_test)

        # Calculate metrics
        metrics = calculate_metrics(y_test, y_pred)

        # Print metrics
        print(f"\nAccuracy:  {metrics['accuracy']:.4f}")
        print(f"Precision: {metrics['precision']:.4f}")
        print(f"Recall:    {metrics['recall']:.4f}")
        print(f"F1-Score:  {metrics['f1_score']:.4f}")

        # Print classification report
        report = print_classification_report(y_test, y_pred, model_name)
```

```
        # Plot confusion matrix
        plot_confusion_matrix(y_test, y_pred, model_name, output_dir)

        # Plot ROC curve
        plot_roc_curve(model, X_test, y_test, model_name, output_dir)

        metrics["classification_report"] = report

        print("=" * 70)

        return metrics

    except Exception as e:
        print(f"[ERROR] Error evaluating {model_name}: {e}")
        return {}


def evaluate_all_models(trained_models, X_test, y_test, output_dir="outputs/charts"):
    """
    Evaluate all trained models.

    Parameters:
        trained_models (dict): Dictionary of trained models.
        X_test (pandas.DataFrame): Testing features.
        y_test (pandas.Series): Testing labels.
        output_dir (str): Output directory for plots.

    Returns:
        dict: Evaluation results for all models.
    """
    try:
        print("\n" + "#" * 70)
        print("# MODEL EVALUATION")
        print("#" * 70)

        all_results = {}

        for name, metadata in trained_models.items():
            model = metadata["model"]
            metrics = evaluate_model(model, X_test, y_test, name, output_dir)
            all_results[name] = metrics

        # Create comparison summary
        print("\n" + "=" * 70)
        print("MODEL COMPARISON SUMMARY")
        print("=" * 70)

        comparison_data = []
        for name, metrics in all_results.items():
            if metrics:
                comparison_data.append({
                    "Model": name,
                    "Accuracy": f"{metrics['accuracy']:.4f}",
                    "Precision": f"{metrics['precision']:.4f}",
                    "Recall": f"{metrics['recall']:.4f}",
                    "F1-Score": f"{metrics['f1_score']:.4f}"
                })

        if comparison_data:
            comparison_df = pd.DataFrame(comparison_data)
            print("\n" + comparison_df.to_string(index=False))

        print("\n" + "#" * 70)
        print("# EVALUATION COMPLETED SUCCESSFULLY")
        print("#" * 70)

        return all_results

    except Exception as e:
        print(f"[ERROR] Error evaluating all models: {e}")
        return {}
```

```
def save_evaluation_report(all_results, output_path):
    """
    Save evaluation report to a text file.

    Parameters:
        all_results (dict): Evaluation results for all models.
        output_path (str): Path to save the report.
    """
    try:
        print(f"\n[Evaluation] Saving report to: {output_path}")

        if not os.path.exists(os.path.dirname(output_path)):
            os.makedirs(os.path.dirname(output_path))

        with open(output_path, "w") as f:
            f.write("=" * 70 + "\n")
            f.write("MODEL EVALUATION REPORT\n")
            f.write("AI Impact on Jobs & Layoff Risk Analysis\n")
            f.write("=" * 70 + "\n\n")

            for name, metrics in all_results.items():
                f.write(f"\n{'=' * 70}\n")
                f.write(f"MODEL: {name}\n")
                f.write(f"{'=' * 70}\n\n")

                if metrics:
                    f.write(f"Accuracy:  {metrics['accuracy']:.4f}\n")
                    f.write(f"Precision: {metrics['precision']:.4f}\n")
                    f.write(f"Recall:    {metrics['recall']:.4f}\n")
                    f.write(f"F1-Score:  {metrics['f1_score']:.4f}\n\n")

                    if "classification_report" in metrics:
                        f.write("Classification Report:\n")
                        f.write(metrics["classification_report"])
                        f.write("\n")

            f.write("\n" + "=" * 70 + "\n")
            f.write("END OF REPORT\n")
            f.write("=" * 70 + "\n")

        print(f"  Report saved successfully!")

    except Exception as e:
        print(f"[ERROR] Error saving evaluation report: {e}")
```

src/prediction.py

```
"""
Prediction Module for AI Impact on Jobs & Layoff Risk Analysis.

This module provides functions to make predictions using trained models.
It includes single prediction and batch prediction capabilities.

Author: Sharma Saurabh (250863131007)
Course: BE05000231 - Python for Data Science (GTU Semester-5)
Month: 3 - Modeling Phase
"""

import pandas as pd
import numpy as np
import joblib
import os


def load_model(model_path):
    """
    Load a trained model from disk.

    Parameters:
        model_path (str): Path to the saved model.

    Returns:
        model: Loaded model object.
    """
    try:
        if not os.path.exists(model_path):
            print(f"[ERROR] Model file not found: {model_path}")
            return None

        model = joblib.load(model_path)
        print(f"[OK] Model loaded from: {model_path}")
        return model

    except Exception as e:
        print(f"[ERROR] Error loading model: {e}")
        return None


def load_preprocessor(scaler_path, encoder_path):
    """
    Load preprocessor objects from disk.

    Parameters:
        scaler_path (str): Path to the saved scaler.
        encoder_path (str): Path to the saved encoder.

    Returns:
        tuple: (scaler, encoders)
    """
    try:
        scaler = None
        encoders = {}

        if os.path.exists(scaler_path):
            scaler = joblib.load(scaler_path)
            print(f"[OK] Scaler loaded from: {scaler_path}")

        if os.path.exists(encoder_path):
            encoders = joblib.load(encoder_path)
            print(f"[OK] Encoders loaded from: {encoder_path}")

        return scaler, encoders

    except Exception as e:
        print(f"[ERROR] Error loading preprocessor: {e}")
        return None, {}
```

```
def predict_single(model, scaler, encoders, input_data, feature_columns):
    """
    Make prediction for a single input.

    Parameters:
        model: Trained model.
        scaler: Fitted scaler (or None).
        encoders (dict): Dictionary of fitted encoders.
        input_data (dict): Input features as dictionary.
        feature_columns (list): List of feature column names.

    Returns:
        dict: Prediction result with class and probabilities.
    """
    try:
        print("\n[Prediction] Making single prediction...")

        # Create DataFrame from input
        input_df = pd.DataFrame([input_data])

        # Ensure correct column order
        for col in feature_columns:
            if col not in input_df.columns:
                input_df[col] = 0

        input_df = input_df[feature_columns]

        # Encode categorical columns
        for col, encoder in encoders.items():
            if col in input_df.columns and col != "layoff_risk":
                try:
                    input_df[col] = encoder.transform(input_df[col].astype(str))
                except ValueError:
                    input_df[col] = 0

        # Scale numerical columns
        if scaler is not None:
            numerical_cols = input_df.select_dtypes(include=["number"]).columns
            input_df[numerical_cols] = scaler.transform(input_df[numerical_cols])

        # Make prediction
        prediction = model.predict(input_df)[0]

        # Get probabilities if available
        probabilities = {}
        if hasattr(model, "predict_proba"):
            proba = model.predict_proba(input_df)[0]
            classes = model.classes_
            for cls, prob in zip(classes, proba):
                probabilities[cls] = round(prob, 4)

        # Decode prediction if encoder available
        if "layoff_risk" in encoders:
            prediction_label = encoders["layoff_risk"].inverse_transform([prediction])[0]
        else:
            prediction_label = prediction

        result = {
            "prediction": prediction_label,
            "probabilities": probabilities
        }

        print(f"\n  Prediction: {prediction_label}")
        if probabilities:
            print("  Probabilities:")
            for cls, prob in probabilities.items():
                if "layoff_risk" in encoders:
                    cls_label = encoders["layoff_risk"].inverse_transform([cls])[0]
                else:
                    cls_label = cls
                print(f"    {cls_label}: {prob*100:.2f}%")

        return result
```

```
    except Exception as e:
        print(f"[ERROR] Error making prediction: {e}")
        return {"prediction": None, "probabilities": {}}


def predict_batch(model, scaler, encoders, df, feature_columns):
    """
    Make predictions for a batch of data.

    Parameters:
        model: Trained model.
        scaler: Fitted scaler (or None).
        encoders (dict): Dictionary of fitted encoders.
        df (pandas.DataFrame): Input DataFrame.
        feature_columns (list): List of feature column names.

    Returns:
        pandas.DataFrame: DataFrame with predictions added.
    """
    try:
        print("\n[Prediction] Making batch predictions...")

        result_df = df.copy()

        # Ensure correct column order
        for col in feature_columns:
            if col not in result_df.columns:
                result_df[col] = 0

        input_df = result_df[feature_columns].copy()

        # Encode categorical columns
        for col, encoder in encoders.items():
            if col in input_df.columns and col != "layoff_risk":
                try:
                    input_df[col] = encoder.transform(input_df[col].astype(str))
                except ValueError:
                    input_df[col] = 0

        # Scale numerical columns
        if scaler is not None:
            numerical_cols = input_df.select_dtypes(include=["number"]).columns
            input_df[numerical_cols] = scaler.transform(input_df[numerical_cols])

        # Make predictions
        predictions = model.predict(input_df)

        # Decode predictions if encoder available
        if "layoff_risk" in encoders:
            predictions_labels = encoders["layoff_risk"].inverse_transform(predictions)
        else:
            predictions_labels = predictions

        result_df["predicted_risk"] = predictions_labels

        # Add probabilities if available
        if hasattr(model, "predict_proba"):
            proba = model.predict_proba(input_df)
            classes = model.classes_
            for i, cls in enumerate(classes):
                if "layoff_risk" in encoders:
                    cls_label = encoders["layoff_risk"].inverse_transform([cls])[0]
                else:
                    cls_label = cls
                result_df[f"prob_{cls_label}"] = proba[:, i].round(4)

        print(f"  Predictions made for {len(result_df)} samples")

        return result_df

    except Exception as e:
        print(f"[ERROR] Error making batch predictions: {e}")
        return df
```

```
def save_predictions(predictions_df, output_path):
    """
    Save predictions to a CSV file.

    Parameters:
        predictions_df (pandas.DataFrame): DataFrame with predictions.
        output_path (str): Path to save the predictions.
    """
    try:
        print(f"\n[Prediction] Saving predictions to: {output_path}")

        if not os.path.exists(os.path.dirname(output_path)):
            os.makedirs(os.path.dirname(output_path))

        predictions_df.to_csv(output_path, index=False)
        print(f"  Predictions saved successfully!")

    except Exception as e:
        print(f"[ERROR] Error saving predictions: {e}")


def run_predictions(model, scaler, encoders, X_test, y_test, config):
    """
    Run complete prediction pipeline.

    Parameters:
        model: Trained model.
        scaler: Fitted scaler.
        encoders (dict): Dictionary of fitted encoders.
        X_test (pandas.DataFrame): Testing features.
        y_test (pandas.Series): Testing labels.
        config: Configuration module.

    Returns:
        pandas.DataFrame: DataFrame with predictions.
    """
    try:
        print("\n" + "#" * 70)
        print("# MAKING PREDICTIONS")
        print("#" * 70)

        # Create test DataFrame with actual values
        test_df = X_test.copy()
        test_df["actual_risk"] = y_test.values

        # Make batch predictions
        predictions_df = predict_batch(
            model, scaler, encoders, test_df, X_test.columns.tolist()
        )

        # Save predictions
        save_predictions(predictions_df, config.PREDICTIONS_PATH)

        # Display sample predictions
        print("\n--- Sample Predictions ---")
        sample_cols = ["actual_risk", "predicted_risk"]
        prob_cols = [col for col in predictions_df.columns if col.startswith("prob_")]
        display_cols = sample_cols + prob_cols
        print(predictions_df[display_cols].head(10))

        print("\n" + "#" * 70)
        print("# PREDICTIONS COMPLETED SUCCESSFULLY")
        print("#" * 70)

        return predictions_df

    except Exception as e:
        print(f"[ERROR] Error in predictions: {e}")
        return None
```

src/utils.py

```
"""
Utility Module for AI Impact on Jobs & Layoff Risk Analysis.

This module provides helper functions used across the project.
It includes file operations, display utilities, and common operations.

Author: Sharma Saurabh (250863131007)
Course: BE05000231 - Python for Data Science (GTU Semester-5)
"""

import os
import pandas as pd
from datetime import datetime


def print_section_header(title):
    """
    Print a formatted section header.

    Parameters:
        title (str): Title to display.
    """
    print("\n" + "#" * 70)
    print(f"# {title}")
    print("#" * 70)


def print_subsection_header(title):
    """
    Print a formatted subsection header.

    Parameters:
        title (str): Title to display.
    """
    print("\n" + "=" * 60)
    print(f"  {title}")
    print("=" * 60)


def display_dataframe_info(df, name="DataFrame"):
    """
    Display comprehensive DataFrame information.

    Parameters:
        df (pandas.DataFrame): DataFrame to display.
        name (str): Name of the DataFrame.
    """
    try:
        print(f"\n--- {name} Information ---")
        print(f"Shape: {df.shape[0]} rows x {df.shape[1]} columns")
        print(f"\nColumns ({len(df.columns)}):")
        for i, col in enumerate(df.columns, 1):
            dtype = df[col].dtype
            non_null = df[col].notna().sum()
            print(f"  {i}. {col} ({dtype}) - {non_null} non-null")

        print(f"\nMemory Usage: {df.memory_usage(deep=True).sum() / 1024:.2f} KB")

    except Exception as e:
        print(f"[ERROR] Error displaying DataFrame info: {e}")


def validate_dataframe(df, required_columns=None):
    """
    Validate DataFrame structure and content.

    Parameters:
        df (pandas.DataFrame): DataFrame to validate.
        required_columns (list): List of required column names.

    Returns:
```

```
        bool: True if valid, False otherwise.
    """
    try:
        if df is None or df.empty:
            print("[VALIDATION] DataFrame is None or empty.")
            return False

        if required_columns:
            missing = [col for col in required_columns if col not in df.columns]
            if missing:
                print(f"[VALIDATION] Missing columns: {missing}")
                return False

        print("[VALIDATION] DataFrame is valid.")
        return True

    except Exception as e:
        print(f"[ERROR] Error validating DataFrame: {e}")
        return False


def ensure_directory(dir_path):
    """
    Ensure directory exists, create if not.

    Parameters:
        dir_path (str): Path to the directory.
    """
    try:
        if not os.path.exists(dir_path):
            os.makedirs(dir_path)
            print(f"Created directory: {dir_path}")

    except Exception as e:
        print(f"[ERROR] Error creating directory: {e}")


def get_file_size(filepath):
    """
    Get file size in human-readable format.

    Parameters:
        filepath (str): Path to the file.

    Returns:
        str: File size as string.
    """
    try:
        if not os.path.exists(filepath):
            return "File not found"

        size_bytes = os.path.getsize(filepath)

        if size_bytes < 1024:
            return f"{size_bytes} B"
        elif size_bytes < 1024 * 1024:
            return f"{size_bytes / 1024:.2f} KB"
        else:
            return f"{size_bytes / (1024 * 1024):.2f} MB"

    except Exception as e:
        return f"Error: {e}"


def get_timestamp():
    """
    Get current timestamp as formatted string.

    Returns:
        str: Formatted timestamp.
    """
    return datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

```
def print_progress_bar(iteration, total, prefix="Progress", suffix="Complete", length=50):
    """
    Print a progress bar.

    Parameters:
        iteration (int): Current iteration.
        total (int): Total iterations.
        prefix (str): Prefix string.
        suffix (str): Suffix string.
        length (int): Length of the bar.
    """
    try:
        percent = f"{100 * (iteration / float(total)):.1f}"
        filled_length = int(length * iteration // total)
        bar = "" * filled_length + "-" * (length - filled_length)
        print(f"\r{prefix} |{bar}| {percent}% {suffix}", end="\r")

        if iteration == total:
            print()

    except Exception as e:
        print(f"[ERROR] Error printing progress bar: {e}")


def save_text_report(content, filepath):
    """
    Save text content to a file.

    Parameters:
        content (str): Text content to save.
        filepath (str): Path to save the file.
    """
    try:
        ensure_directory(os.path.dirname(filepath))

        with open(filepath, "w", encoding="utf-8") as f:
            f.write(content)

        print(f"[OK] Report saved: {filepath}")

    except Exception as e:
        print(f"[ERROR] Error saving report: {e}")


def load_text_report(filepath):
    """
    Load text content from a file.

    Parameters:
        filepath (str): Path to the file.

    Returns:
        str: File content.
    """
    try:
        if not os.path.exists(filepath):
            print(f"[ERROR] File not found: {filepath}")
            return None

        with open(filepath, "r", encoding="utf-8") as f:
            content = f.read()

        return content

    except Exception as e:
        print(f"[ERROR] Error loading report: {e}")
        return None


def format_dict_for_display(data, indent=2):
    """
    Format dictionary for clean display.

    Parameters:
```

```
        data (dict): Dictionary to format.
        indent (int): Indentation level.

    Returns:
        str: Formatted string.
    """
    try:
        lines = []
        prefix = " " * indent

        for key, value in data.items():
            if isinstance(value, dict):
                lines.append(f"{prefix}{key}:")
                lines.append(format_dict_for_display(value, indent + 2))
            elif isinstance(value, list):
                lines.append(f"{prefix}{key}: {', '.join(map(str, value))}")
            else:
                lines.append(f"{prefix}{key}: {value}")

        return "\n".join(lines)

    except Exception as e:
        return f"Error formatting dict: {e}"


def create_project_summary():
    """
    Create a project summary dictionary.

    Returns:
        dict: Project summary.
    """
    summary = {
        "Project Title": "AI Impact on Jobs & Layoff Risk Analysis",
        "Student": "Sharma Saurabh",
        "Enrollment": "250863131007",
        "Course": "BE05000231 - Python for Data Science",
        "Semester": "GTU Semester-5",
        "Duration": "3 Months",
        "Phases": {
            "Month-1": "Foundation Phase - Project Setup, Data Loading, Data Cleaning",
            "Month-2": "Analysis Phase - EDA, Visualization, Feature Engineering",
            "Month-3": "Modeling Phase - ML Training, Evaluation, Prediction"
        }
    }
    return summary
```

README.md

```
# AI Impact on Jobs & Layoff Risk Analysis using Machine Learning

## Project Overview

This project analyzes the impact of Artificial Intelligence on employment and assesses layoff risk factors usi
ng Machine Learning techniques. It is developed as part of the GTU Semester-5 course **BE05000231 - Python for
 Data Science**.

**Duration:** 3 Months (Currently in Month-1 Foundation Phase)

---

## Problem Statement

The rapid advancement of AI technology is transforming the job market, creating both opportunities and challen
ges. Understanding which jobs are at risk of being automated and identifying factors that contribute to layoff
 risk is crucial for workforce planning, career development, and policy making.

This project aims to:
- Analyze job characteristics and AI adoption patterns
- Identify key factors contributing to layoff risk
- Build predictive models (in future phases) to assess job vulnerability to AI automation

---

## Objectives

### Month-1 (Foundation Phase) - Current
- [x] Create professional project structure
- [x] Configure project settings and paths
- [x] Load and validate dataset
- [x] Inspect dataset structure and quality
- [x] Perform basic data cleaning
- [x] Save cleaned dataset for future analysis

### Month-2 (Analysis Phase) - Upcoming
- [ ] Exploratory Data Analysis (EDA)
- [ ] Data Visualization (Histograms, Heatmaps, Pie Charts)
- [ ] Feature Engineering and Encoding
- [ ] Data Preprocessing and Scaling

### Month-3 (Modeling Phase) - Upcoming
- [ ] Machine Learning Model Training
- [ ] Model Evaluation and Comparison
- [ ] Prediction System Development
- [ ] Final Documentation and Reports

---

## Dataset Description

**File:** `ai-impact-jobs-layoff-risk-dataset.csv`

| Column | Description | Type |
|--------|-------------|------|
| Age | Age of the employee | Numeric |
| Education_Level | Highest education attained | Categorical |
| Years_of_Experience | Professional experience in years | Numeric |
| Industry | Industry of employment | Categorical |
| Job_Role | Specific job title/role | Categorical |
| Company_Size | Size of the company (Small/Medium/Large) | Categorical |
| Job_Level | Seniority level (Entry/Mid/Senior) | Categorical |
| Routine_Task_Percentage | Percentage of routine tasks in job | Numeric |
| Creativity_Requirement | Level of creativity needed | Numeric |
| Human_Interaction_Level | Level of human interaction required | Numeric |
| AI_Adoption_Level | Current AI adoption in the role | Categorical |
| Number_of_AI_Tools_Used | Count of AI tools being used | Numeric |
| AI_Usage_Hours_Per_Week | Weekly AI usage hours | Numeric |
| Tasks_Automated_Percentage | Percentage of tasks that can be automated | Numeric |
| AI_Training_Hours | Hours spent on AI training | Numeric |
| Layoff_Risk | Risk level of being laid off (Target) | Categorical |
```

Dataset Size: 20,000 rows x 16 columns

Folder Structure

```
AI-Impact-Jobs-Layoff-Risk/

 README.md                 # Project documentation
 requirements.txt          # Python dependencies
 main.py                   # Main entry point
 config.py                 # Configuration settings

 data/
    raw/                  # Original, unmodified data
       ai-impact-jobs-layoff-risk-dataset.csv
    processed/            # Cleaned and processed data

 src/                      # Source code modules
    __init__.py           # Package initializer
    data_loading.py       # Data loading functions
    data_cleaning.py      # Data cleaning functions

 outputs/
    processed_data/       # Saved cleaned datasets

 notebooks/                # Jupyter notebooks (future use)

 docs/                     # Project documentation
    month1_documentation.md

 tests/                    # Unit tests (future use)
```

Installation

Prerequisites

- Python 3.8 or higher
- pip package manager

Setup Steps

1. **Clone or download the project**
   ```bash
   cd AI-Impact-Jobs-Layoff-Risk
   ```

2. **Install required packages**
   ```bash
   pip install -r requirements.txt
   ```

3. **Verify dataset location**
   Ensure the CSV file is in `data/raw/` folder:
   ```
   data/raw/ai-impact-jobs-layoff-risk-dataset.csv
   ```

4. **Run the project**
   ```bash
   python main.py
   ```

Usage

Running Month-1 Pipeline

```bash
python main.py
```

This will:
1. Load the raw dataset
2. Display dataset information (shape, columns, data types, missing values)
3. Clean the dataset (handle missing values, remove duplicates, standardize columns)
4. Save the cleaned dataset to `outputs/processed_data/cleaned_dataset.csv`

Output

After running, you will find:
- `outputs/processed_data/cleaned_dataset.csv` - The cleaned dataset ready for analysis

Month-1 Scope

This phase focuses on establishing a solid project foundation:

- **Project Structure:** Professional folder organization
- **Configuration:** Centralized path management
- **Data Loading:** Robust CSV loading with error handling
- **Data Inspection:** Comprehensive dataset analysis
- **Data Cleaning:** Missing value handling, duplicate removal, validation
- **Documentation:** Complete project documentation

What's NOT Included (Future Months)
- Machine Learning models
- Data visualization
- Feature engineering
- Predictive analytics
- Web dashboards
- Model evaluation metrics

Code Quality

- **PEP-8 Compliance:** Follows Python coding standards
- **Modular Design:** Reusable functions and clear separation of concerns
- **Error Handling:** Comprehensive exception handling throughout
- **Documentation:** Professional docstrings and inline comments
- **Configuration:** No hardcoded paths, centralized settings

---

Author

Student ID: BE05000231
Course: Python for Data Science (GTU Semester-5)
Project Duration: 3 Months

License

This project is developed for educational purposes as part of GTU Semester-5 coursework.